

OBJETIVO

Esta é a etapa final do SGCM. Nas etapas anteriores, o sistema ganhou sua estrutura de dados, sua API RESTful e sua camada de segurança. Nesta etapa, o sistema ganha sua razão de existir do ponto de vista clínico: os atendimentos são realizados, os procedimentos são registrados, os prontuários são gerados e os laudos são emitidos — documentos formais que representam o resultado concreto de cada interação entre médico e paciente.

Esta etapa é a mais densa em modelagem de domínio. Duas novas hierarquias de herança são introduzidas — `Appointment` e `Procedure` — cada uma com subclasses que têm atributos, regras e ciclos de vida distintos. O sistema precisa garantir a integridade dessas entidades, respeitar as dependências entre elas e produzir documentos clínicos — prontuários e laudos — que reflitam com precisão o que foi registrado.

Ao final desta etapa, o sistema deverá ser capaz de:

- **Registrar atendimentos clínicos** — consultas, exames e retornos gerados a partir de agendamentos confirmados, cada um com seu ciclo de vida controlado e suas regras específicas;
- **Gerenciar procedimentos médicos** — simples e especializados, com controle de autorização para os que exigem aprovação prévia do administrador;
- **Emitir prontuários** — registros clínicos formais gerados ao encerramento de qualquer atendimento, acessíveis apenas pelos perfis autorizados;
- **Emitir e validar laudos** — documentos formais em PDF gerados para atendimentos do tipo Exame, com código único de validação e *endpoint* público de verificação de autenticidade;
- **Revogar laudos** — com rastreabilidade completa de quem revogou e por quê;
- **Gerar relatórios administrativos** — métricas sobre agendamentos, atendimentos e procedimentos, acessíveis apenas ao perfil Admin;
- **Completar a documentação da API** — documentação Swagger final, abrangendo todos os *endpoints* do sistema com o nível de detalhe esperado de uma API em produção.

O que se avalia nesta etapa não é apenas se as entidades foram criadas corretamente. É se o grupo compreende as relações entre elas, se as regras de negócio do domínio clínico foram respeitadas, se a segurança implementada na Etapa 2 foi estendida corretamente para os novos *endpoints* e se a documentação final representa fielmente o sistema construído ao longo do semestre.

REQUISITOS GERAIS DESTA ETAPA

Nesta terceira e última etapa, o SGCM atinge sua completude funcional. O sistema deixa de ser uma API que gerencia agendamentos e usuários para se tornar um sistema clínico completo — capaz de registrar o que acontece dentro de uma consulta, documentar formalmente os resultados de exames e oferecer à administração da clínica uma visão consolidada do que está acontecendo. Tudo isso sobre uma base já construída: a autenticação, a autorização e a infraestrutura transversal das etapas anteriores estão prontas e devem ser aplicadas naturalmente a cada novo *endpoint* desta entrega.

A diferença desta etapa em relação às anteriores não está apenas na quantidade de entidades novas — está na complexidade das relações entre elas. Um laudo depende de um exame. Um exame é um tipo de atendimento. Um atendimento depende de um agendamento confirmado. Um procedimento especializado depende de uma autorização do administrador antes de poder ser realizado. Essas dependências formam uma cadeia que o sistema deve respeitar em cada operação — e o grupo deve compreender essa cadeia integralmente antes de escrever a primeira linha de código desta etapa.

De forma geral, o sistema deve contemplar nesta etapa:

- **Gestão de atendimentos:** o sistema deve permitir o registro de atendimentos clínicos nas três modalidades — consulta, exame e retorno — gerados exclusivamente a partir de agendamentos confirmados. Cada modalidade carrega atributos específicos e segue regras distintas. O ciclo de vida de um atendimento — iniciado e encerrado — deve ser controlado com rigor, pois seu estado determina o que pode ser feito com as entidades que dele dependem: um prontuário só pode ser criado após o atendimento ser encerrado, um laudo só pode ser emitido após o exame ser encerrado com resultado registrado.
- **Gestão de procedimentos:** procedimentos médicos são realizados durante atendimentos em andamento. Procedimentos simples podem ser registrados diretamente. Procedimentos especializados que exigem autorização prévia devem ter seu ciclo de autorização controlado — com o administrador como responsável pela aprovação ou negação — e só podem ser considerados realizados após a autorização ser concedida. O grupo deve pensar nas implicações clínicas de cada estado desse ciclo e garantir que o sistema as respeite.
- **Prontuários:** o prontuário é o registro clínico oficial de um atendimento encerrado. Ele não pode ser excluído — representa um documento médico permanente. Atualizações são permitidas, mas devem ser rastreadas. O acesso deve ser cuidadosamente controlado por perfil e por recurso — nem todo usuário autenticado pode ver qualquer prontuário.
- **Laudos:** o laudo é o documento mais formal do sistema. Gerado exclusivamente para exames encerrados com resultado registrado, ele deve ser disponibilizado em PDF, possuir um código único de validação e ser verificável publicamente — sem autenticação — por qualquer pessoa que tenha o código. Esse *endpoint* público é o único além dos de *login* e *refresh* que dispensa autenticação, e sua existência tem uma motivação clínica clara: permitir que terceiros — seguradoras, outros médicos, instituições — verifiquem a autenticidade de um laudo sem precisar de acesso ao sistema.
- **Relatórios administrativos:** o administrador precisa de visibilidade sobre o funcionamento da clínica. Os relatórios consolidam métricas sobre agendamentos, atendimentos, procedimentos e ocupação da agenda dos médicos. Eles não criam nem alteram dados — apenas consultam e agregam o que já existe. A lógica de agregação, no entanto, pode ser complexa e deve residir nos *services* — nunca nos *controllers*.

- **Documentação final:** esta é a última entrega. O Swagger deve cobrir todos os *endpoints* do sistema — das três etapas — com o nível de detalhe esperado de uma API em produção. Não é aceitável que *endpoints* das Etapas 1 ou 2 estejam desatualizados ou com exemplos genéricos na entrega final.

APLICAÇÃO DA INFRAESTRUTURA DAS ETAPAS ANTERIORES

Toda a infraestrutura construída nas etapas anteriores se aplica automaticamente aos novos *endpoints* desta etapa — desde que tenha sido registrada globalmente. O *middleware* de *logging* registrará todas as requisições. O *Transform Interceptor* envolverá todas as respostas de sucesso. O *Exception Filter* formatará todos os erros no padrão RFC 7807. Os *guards* de autenticação e autorização protegerão todos os *endpoints* — exceto os explicitamente marcados com `@Public()`.

O grupo deve verificar se essa aplicação automática está funcionando corretamente para os novos *endpoints* antes de declarar qualquer funcionalidade como concluída. Um *endpoint* que não passa pelo *middleware* de *logging*, que não tem sua resposta envolvida pelo *interceptor* ou que não está protegido pelo *guard* é um *endpoint* implementado de forma incompleta — independentemente de sua lógica de negócio estar correta.



REFLEXÃO

Há algo na infraestrutura das etapas anteriores que precisa ser ajustado para esta etapa?

A infraestrutura foi construída pensando nos *endpoints* das Etapas 1 e 2. Com os novos *endpoints* desta etapa — em particular o *endpoint* público de validação de laudos e o *endpoint* de *download* de PDF — podem surgir casos que a infraestrutura não tratava anteriormente. O *Transform Interceptor* foi projetado para envolver respostas JSON — o que acontece quando o *endpoint* retorna um *buffer* de PDF? O *middleware* de *logging* registra corretamente requisições para *endpoints* que retornam arquivos? O grupo deve identificar esses casos de borda, testar o comportamento da infraestrutura existente com os novos tipos de resposta e ajustar o que for necessário. Registre no relatório o que foi ajustado e por quê.

PAGINAÇÃO E FILTROS

Os parâmetros de paginação e o formato de resposta definidos na Etapa 1 permanecem inalterados e devem ser aplicados a todos os *endpoints* de listagem desta etapa. O grupo deve garantir consistência total — os mesmos parâmetros `page`, `limit` e `sort`, o mesmo envelope `{ data, meta }` com os mesmos campos de metadados.

Adicionalmente, os *endpoints* de listagem desta etapa introduzem filtros mais complexos do que os das etapas anteriores. Listar atendimentos por médico, por paciente, por tipo, por status e por intervalo de datas — possivelmente combinados — exige uma estratégia de filtragem dinâmica no repositório que o grupo deve implementar de forma limpa e reutilizável.



REFLEXÃO

Como implementar filtros dinâmicos de forma reutilizável?

Os *endpoints* de listagem desta etapa aceitam múltiplos filtros opcionais que podem ser combinados em qualquer permutação. Uma abordagem é construir a *query* dinamicamente no *service* usando o `QueryBuilder` do TypeORM, adicionando condições apenas quando o filtro estiver presente. Outra é criar um objeto de critérios de busca — um DTO de *query* — que encapsula todos os filtros e é passado para um método do repositório que constrói a *query* correspondente. O grupo deve avaliar qual abordagem produz código mais legível e mais fácil de manter, considerando que o padrão escolhido será aplicado em múltiplos *endpoints*. A decisão deve ser consistente em todos os *endpoints* de listagem desta etapa e justificada no relatório.

VALIDAÇÃO E DTOS DESTA ETAPA

Os DTOs desta etapa têm características especiais que o grupo deve considerar com cuidado:

- **DTOs com herança:** assim como nas etapas anteriores, cada subtipo de `Appointment` e `Procedure` tem atributos específicos obrigatórios que não fazem sentido para os outros. O grupo deve decidir como estruturar os DTOs para essas hierarquias — a mesma decisão tomada para `Schedule` na Etapa 1 deve ser revisitada aqui, possivelmente com o benefício da experiência acumulada.
- **Validações de estado:** alguns campos só são válidos em determinados estados. O `result` de um `Exam` pode ser `null` no momento da criação e preenchido posteriormente. O `diagnosticHypothesis` de uma `Consultation` é opcional. O `originAppointmentId` de um `FollowUp` é obrigatório e deve referenciar um atendimento existente do mesmo paciente. Essas validações vão além do formato — são validações de domínio que pertencem ao *service*.
- **DTO de revogação:** o `PATCH /reports/{id}/revoke` recebe um `revokedReason` obrigatório. O grupo deve criar um DTO específico para essa operação — não reutilizar um DTO genérico de atualização.



REFLEXÃO

Como o grupo lidou com DTOs de hierarquias nas etapas anteriores, e o que aprendeu?

Na Etapa 1, o grupo definiu uma estratégia para estruturar os DTOs da hierarquia de `Schedule`: utilizar um único DTO com campos opcionais por modalidade ou criar DTOs separados para cada subtipo. Essa escolha impactou diretamente a validação, a documentação no Swagger e a clareza do código.

Agora, ao lidar com `Appointment` e `Procedure`, é importante revisitar essa decisão com senso crítico:

- A abordagem adotada anteriormente funcionou bem na prática?
- Gerou dificuldades de manutenção, validação ou documentação?
- Faz sentido reaplicar o mesmo padrão neste contexto?
- Ou existem razões técnicas para adotar uma estratégia diferente?

A análise deve refletir aprendizado real ao longo do projeto. Registre no relatório não apenas a decisão atual, mas também a avaliação da escolha feita na Etapa 1, destacando o que funcionou, o que não funcionou e por quê.

Evite justificativas retroativas: a resposta deve evidenciar evolução de entendimento e maturidade arquitetural.

ESTRUTURA DO SISTEMA — VISÃO GERAL

Esta é a etapa de maior riqueza em termos de domínio clínico. Enquanto as etapas anteriores construíram a infraestrutura e a segurança do sistema, esta etapa adiciona as entidades que representam o coração do trabalho de uma clínica médica — o que acontece dentro de um atendimento, o que é registrado, o que é documentado e o que é reportado. Quatro novos módulos de domínio são introduzidos, cada um com suas próprias entidades, regras de negócio e dependências.

Antes de criar qualquer módulo ou escrever qualquer linha de código, o grupo deve ter uma visão clara de como essas novas partes se encaixam no sistema já construído. Os diagramas de classes desta etapa devem ser consultados em profundidade.

Figura 1 — Diagrama de classe de atendimentos e procedimentos

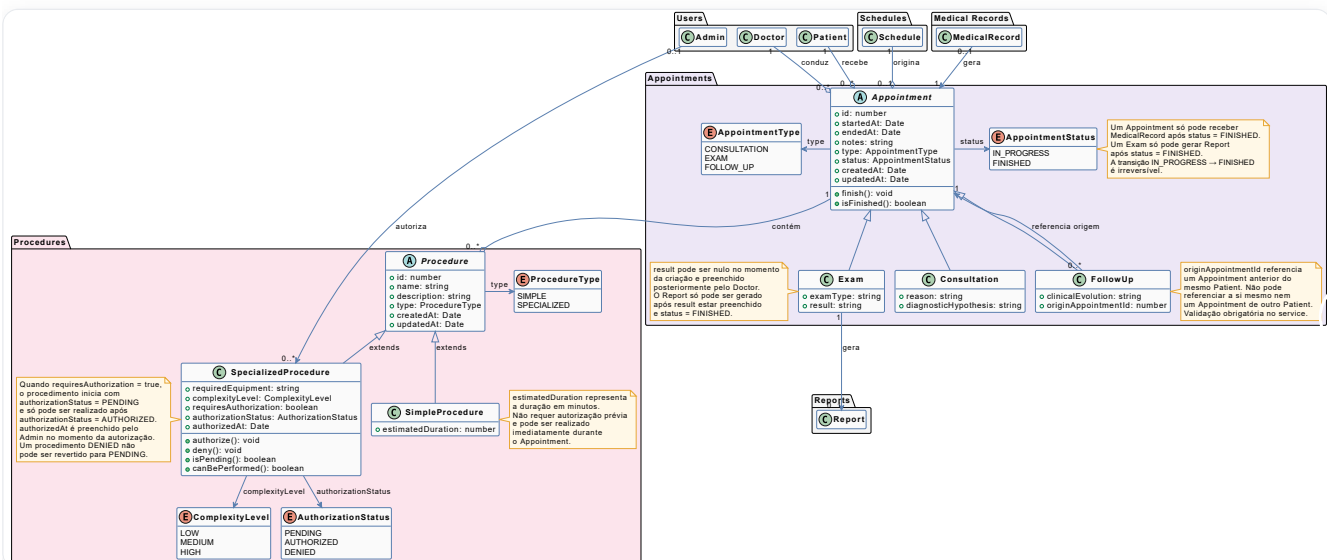
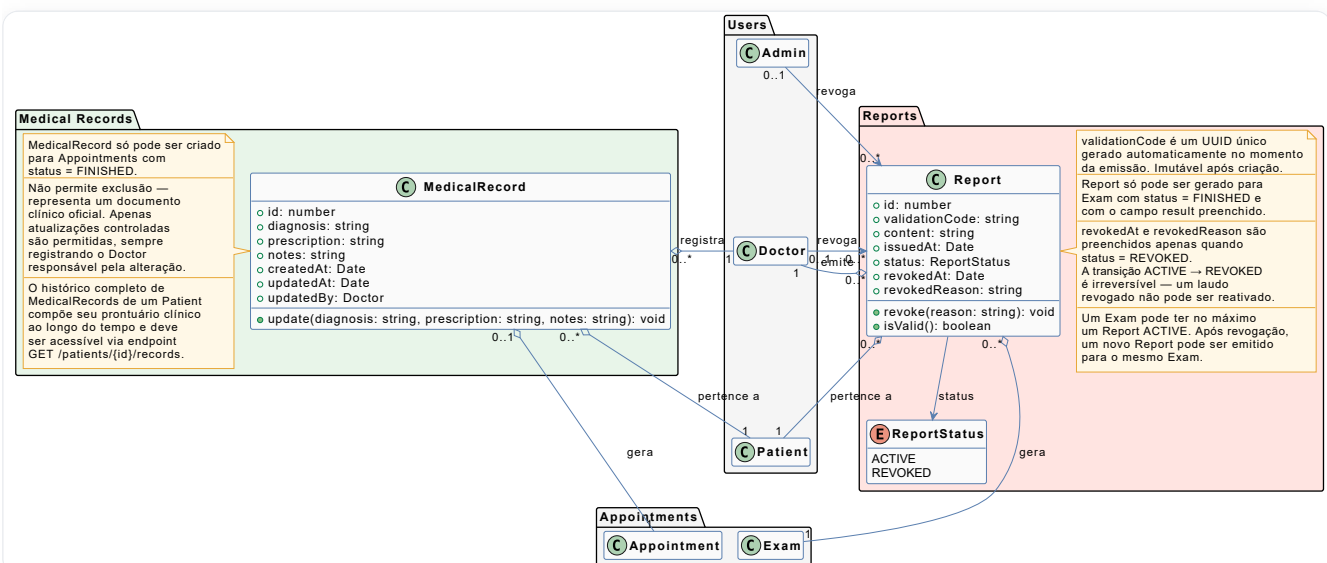


Figura 2 — Diagrama de classe de prontuário e laudo



OS NOVOS MÓDULOS

AppointmentsModule é o módulo central desta etapa. Ele gerencia o ciclo de vida dos atendimentos clínicos — desde a criação a partir de um agendamento confirmado até o encerramento que habilita a criação do prontuário e a emissão do laudo. Sua posição na arquitetura é estratégica: ele depende do **SchedulesModule** para verificar e atualizar o agendamento de origem, e é dependência dos módulos de prontuário e laudo, que só operam sobre atendimentos encerrados.

ProceduresModule gerencia os procedimentos realizados durante atendimentos. É um módulo relativamente coeso — seus dados raramente precisam ser acessados de forma independente do atendimento ao qual pertencem. A complexidade reside no ciclo de autorização dos procedimentos especializados, que envolve o **Admin** como ator externo ao fluxo médico.

MedicalRecordsModule gerencia os prontuários clínicos. É intencionalmente simples em termos de operações — criação, atualização e consulta — mas rico em regras de acesso. O controle por recurso aqui é especialmente crítico: prontuários contêm informações clínicas sensíveis que não devem ser acessíveis a qualquer usuário autenticado.

ReportsModule gerencia os laudos — documentos formais gerados para exames encerrados. Sua singularidade no sistema é o *endpoint* público de validação, que quebra o padrão de autenticação obrigatória de todos os demais *endpoints*. A geração de PDF adiciona uma dimensão técnica que vai além do CRUD convencional.

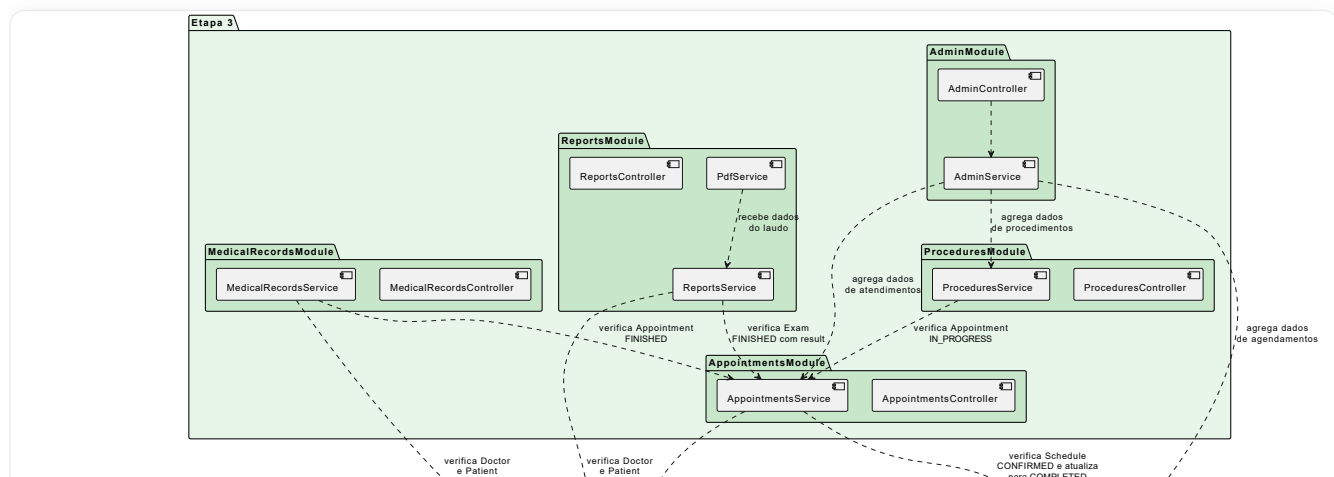
Além desses quatro módulos, um quinto componente — que o grupo deve decidir como organizar — consolida os relatórios administrativos. Esses relatórios agregam dados de múltiplos módulos e são exclusivos do **Admin**.

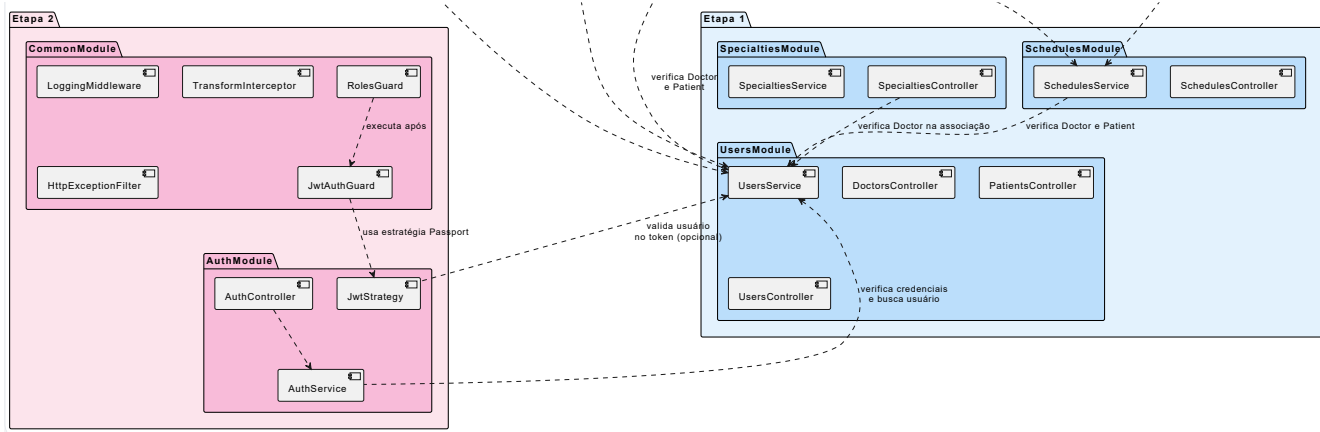
MAPEANDO AS DEPENDÊNCIAS

Com quatro novos módulos e múltiplas dependências entre eles e os módulos existentes, o risco de dependências circulares e de acoplamento indevido é o maior do projeto até agora. O grupo deve mapear essas dependências antes de criar qualquer módulo.

A figura a seguir ilustra as dependências esperadas entre os módulos desta etapa e os módulos existentes:

Figura 3 — Diagrama de dependências





Cada seta representa uma dependência — o módulo à esquerda precisa de algo do módulo à direita. O grupo deve verificar se há setas no sentido oposto que criariam ciclos — e eliminar essas dependências circulares antes de implementar.



REFLEXÃO

O diagrama de dependências anterior está correto e completo?

O diagrama apresentado anteriormente é apenas uma representação simplificada das dependências esperadas. Cabe ao grupo analisá-lo de forma crítica e validar se ele realmente reflete o domínio que está sendo implementado.

Durante essa análise, considere os seguintes pontos:

- Há dependências importantes que não foram representadas no diagrama?
- Existe alguma dependência que parece inadequada ou inconsistente com o domínio?

Por exemplo:

- O `MedicalRecordsModule` depende de informações do `AppointmentsModule` — isso faz sentido no contexto do sistema?
- O inverso também ocorre? Ou seja, o `AppointmentsModule` precisa de algo do `MedicalRecordsModule`?
- Se houver dependência bidirecional, como ela será resolvida sem comprometer a arquitetura (por exemplo, evitando acoplamento excessivo ou ciclos)?

Antes de iniciar a implementação, o grupo deve construir seu próprio mapa de dependências entre módulos, com base nas necessidades reais do sistema.

Esse diagrama deve ser:

- Registrado no relatório;
- Utilizado como guia durante o desenvolvimento;
- E validado ao final da entrega, verificando se a implementação permaneceu fiel à arquitetura planejada.

A POSIÇÃO DE CADA COMPONENTE NO PIPELINE

Os componentes de infraestrutura implementados nas etapas anteriores continuam ativos e se aplicam automaticamente a todos os novos *endpoints*. No entanto, esta etapa introduz dois tipos de *endpoint* que se comportam de forma diferente do padrão estabelecido — e o grupo deve verificar como a infraestrutura existente se comporta nesses casos.

O *endpoint* público de validação de laudos (`GET /reports/validate/{code}`) quebra o padrão de autenticação obrigatória. Ele deve ser marcado com `@Public()` — o decorator implementado na Etapa 2 — para que o `JwtAuthGuard` o ignore. O *Transform Interceptor* e o *middleware* de *logging* devem continuar funcionando normalmente para esse *endpoint*. O grupo deve verificar se o decorator `@Public()` implementado anteriormente suporta esse novo caso de uso sem modificação.

O **endpoint de download de PDF** (`GET /reports/{id}/pdf`) retorna um *buffer* binário — não JSON. O *Transform Interceptor* foi projetado para envolver respostas JSON no envelope `{ data, meta }`. O que acontece quando o *interceptor* recebe um *buffer* de PDF? O grupo deve investigar esse comportamento e implementar uma solução — seja adaptando o *interceptor* para detectar respostas binárias, seja usando um mecanismo do NestJS para que o *endpoint* sinalize que sua resposta não deve ser transformada.



REFLEXÃO

Como o *Transform Interceptor* deve se comportar com respostas não-JSON?

O *Transform Interceptor* foi projetado na Etapa 2 assumindo que todas as respostas de sucesso são objetos JavaScript que podem ser serializados como JSON. O *endpoint* de PDF quebra essa suposição. O grupo deve investigar como o NestJS lida com respostas binárias — `StreamableFile`, `Buffer`, `Response` direto — e como o *interceptor* pode identificar que aquela resposta não deve ser transformada. Uma abordagem é verificar o tipo do valor retornado pelo *handler*; outra é usar um decorator como `@SkipTransform()` que o *interceptor* reconhece. O grupo deve escolher uma abordagem, verificar se ela é consistente com as decisões tomadas na Etapa 2 e justificá-la no relatório.

HERANÇA NO BANCO DE DADOS — REVISÃO E NOVAS HIERARQUIAS

Esta etapa introduz as duas hierarquias de herança mais ricas do sistema em termos de atributos e regras de negócio. O grupo já implementou herança para `User` e `Schedule` nas etapas anteriores — e agora tem experiência real com as implicações de cada estratégia.

Antes de escolher a estratégia para `Appointment` e `Procedure`, o grupo deve visitar as escolhas anteriores com olhar crítico:

- A estratégia adotada para `User` facilitou ou dificultou as consultas de listagem com filtro por tipo?
- A estratégia adotada para `Schedule` gerou *queries* eficientes quando precisou retornar agendamentos de todas as modalidades misturados?
- Houve problemas com o SQLite em relação às estratégias escolhidas — constraints não suportadas, comportamentos inesperados?

Essas experiências devem informar as decisões para `Appointment` e `Procedure`. O grupo não precisa ser consistente com as escolhas anteriores — mas precisa justificar cada nova escolha com base nos critérios do domínio e no aprendizado acumulado.

**REFLEXÃO****O que a experiência das etapas anteriores ensina sobre herança no banco de dados?**

Esta é a terceira vez que o grupo implementa herança no TypeORM com SQLite. Ao longo das etapas anteriores, o grupo provavelmente encontrou situações que não estavam previstas — comportamentos inesperados do ORM, limitações do SQLite, dificuldades em consultas complexas. O grupo deve registrar no relatório o que aprendeu sobre herança no banco de dados ao longo do projeto — não apenas as decisões técnicas, mas as lições que levaria para um próximo projeto. Essa reflexão retrospectiva é um dos elementos que diferencia o relatório final de um simples registro de decisões.

ORGANIZAÇÃO DOS RELATÓRIOS ADMINISTRATIVOS

Os relatórios administrativos são um caso especial na arquitetura desta etapa. Eles não gerenciam entidades próprias — apenas consultam e agregam dados de múltiplos módulos. Isso levanta uma questão de organização que o grupo deve resolver conscientemente.

Uma abordagem é criar um `AdminModule` dedicado, que importa os *services* dos módulos de domínio e os usa apenas para consultas de agregação. Essa abordagem mantém a lógica de relatório centralizada e separada das responsabilidades de cada módulo de domínio.

Outra abordagem é distribuir a lógica de relatório nos próprios módulos de domínio — cada módulo expõe um método de agregação que o *controller* de administração chama diretamente. Essa abordagem mantém a coesão de cada módulo, mas dispersa a lógica de relatório.

Uma terceira abordagem é usar um padrão de repositório de leitura — separando as *queries* de leitura para relatórios das operações de escrita do domínio, o que facilita otimizações futuras sem afetar o fluxo principal.

**REFLEXÃO****Qual é a abordagem mais adequada para os relatórios administrativos no contexto do SGCM?**

Cada uma das abordagens acima tem *trade-offs* em termos de coesão, acoplamento e facilidade de manutenção. O grupo deve avaliar qual se encaixa melhor na arquitetura já construída — considerando como os módulos foram organizados nas etapas anteriores e quais dependências já existem. Uma abordagem que funciona bem para quatro relatórios pode não escalar para dez. O grupo deve pensar prospectivamente: se a clínica solicitasse cinco novos relatórios amanhã, qual abordagem exigiria menos refatoração? Registre a decisão e seu raciocínio no relatório.

BOAS PRÁTICAS DESTA ETAPA

Esta etapa consolida tudo que foi construído anteriormente. As fronteiras entre camadas — *controller* recebe e responde, *service* decide e executa, repositório persiste e consulta — devem ser mantidas com o mesmo rigor. A autenticação e a autorização devem cobrir todos os novos *endpoints*. O formato RFC 7807 deve ser aplicado a todos os novos cenários de erro.

Mas há uma boa prática específica desta etapa que merece atenção especial: **a integridade dos dados clínicos**. Prontuários e laudos são documentos médicos que não podem ser excluídos — apenas revogados, no caso dos laudos. Procedimentos realizados são parte do registro clínico de um atendimento encerrado. O grupo deve implementar proteções explícitas contra operações que comprometeriam a integridade desses registros, retornando erros claros e descritivos quando uma tentativa indevida for feita.



REFLEXÃO

Quais operações desta etapa têm maior potencial de comprometer a integridade dos dados clínicos?

Ao longo desta etapa, o grupo implementará operações que têm efeitos irreversíveis ou que dependem do estado de outras entidades. Encerrar um atendimento é irreversível. Emitir um laudo gera um documento permanente. Revogar um laudo é irreversível. Negar um procedimento especializado é definitivo. O grupo deve identificar todas as operações com essas características, garantir que o sistema as trate com o cuidado adequado — validando o estado das entidades envolvidas antes de executar — e documentar no relatório como cada uma foi protegida. Pense também nos casos de borda: o que acontece se dois usuários tentarem encerrar o mesmo atendimento simultaneamente?

HERANÇA NO BANCO DE DADOS — NOVAS HIERARQUIAS

Esta é a terceira vez que o grupo enfrenta a decisão de estratégia de herança no banco de dados. Nas etapas anteriores, essa decisão foi tomada com base em critérios teóricos e nas características do domínio. Agora, o grupo tem algo que não tinha antes: experiência real com as implicações de cada estratégia no TypeORM com SQLite. Essa experiência deve informar — e possivelmente questionar — as decisões desta etapa.

Antes de escolher qualquer estratégia, o grupo deve revisitar honestamente o que funcionou e o que não funcionou nas hierarquias anteriores. Houve problemas de performance em consultas que retornavam registros de todos os subtipos misturados? A estratégia escolhida para `User` facilitou ou complicou a filtragem por tipo? O SQLite apresentou limitações que não eram esperadas? Essas respostas devem guiar as escolhas desta etapa — e ser documentadas no relatório como parte da evolução de aprendizado do grupo ao longo do projeto.

`Appointment` → `Consultation`, `Exam`, `FollowUp`

O atendimento é a entidade de maior centralidade clínica desta etapa. Toda a cadeia de documentação — prontuário, laudo — parte de um `Appointment`. Sua hierarquia possui três subclasses com características bem distintas:

- `Consultation` carrega `reason` — o motivo pelo qual o paciente procurou atendimento — e `diagnosticHypothesis` — a hipótese diagnóstica levantada pelo médico ao final da consulta. O `reason` é obrigatório no momento da criação; o `diagnosticHypothesis` pode ser preenchido posteriormente, durante o atendimento.
- `Exam` carrega `examType` — o tipo de exame realizado, como eletrocardiograma ou hemograma — e `result` — o resultado do exame. O `result` é o campo mais crítico desta subclasse: ele é opcional no momento da criação, pode ser preenchido durante ou após o atendimento, e sua presença é condição obrigatória para a emissão do laudo. Um exame encerrado sem resultado não pode gerar laudo.
- `FollowUp` carrega `clinicalEvolution` — a evolução clínica observada em relação ao atendimento anterior — e `originAppointmentId` — a referência ao atendimento que originou este retorno. Essa referência é a mais complexa do sistema: ela deve apontar para um `Appointment` anterior do mesmo `Patient`, não pode referenciar a si mesma e não pode referenciar um atendimento de outro paciente. Essas validações devem ocorrer no `service` antes da persistência.

PERFIL DE CONSULTAS DA HIERARQUIA `Appointment`

Para escolher a estratégia de herança adequada, o grupo deve analisar os padrões de consulta que a aplicação fará sobre esta hierarquia. Diferentemente de `User` — onde a consulta por tipo específico é frequente — e de `Schedule` — onde a listagem geral é o caso mais comum — `Appointment` tem um perfil misto:

- O sistema frequentemente precisará buscar **todos os atendimentos** de um médico ou paciente, independentemente do tipo — para listagens gerais e histórico clínico. Esse padrão favorece estratégias que não exigem `UNION` entre tabelas.
- O sistema também precisará buscar **especificamente exames** — para listar candidatos à emissão de laudo,

verificar se um exame tem resultado preenchido, ou filtrar atendimentos por tipo em relatórios administrativos. Esse padrão é eficiente em qualquer estratégia, desde que o discriminador de tipo seja indexado.

- O sistema raramente precisará buscar os atributos específicos de uma subclasse sem já saber o tipo — ou seja, raramente buscará `diagnosticHypothesis` de todas as consultas sem antes filtrar pelo tipo `CONSULTATION`.



REFLEXÃO

Como o perfil de consultas da hierarquia `Appointment` influencia a escolha da estratégia?

Com base na análise realizada, o grupo deve avaliar criticamente cada estratégia de modelagem disponível — *Single Table*, *Joined Table* e *Concrete Table* — levando em conta, sobretudo, os padrões de consulta identificados no sistema.

Para orientar essa decisão, considere as seguintes questões:

- Qual estratégia gera as consultas mais eficientes para o caso de uso mais frequente: buscar todos os atendimentos de um médico?
- Qual abordagem facilita a filtragem por tipo, especialmente para a emissão de laudos?
- Qual estratégia oferece mais clareza e segurança na validação do `originAppointmentId` no `FollowUp`, que precisa verificar tanto o tipo quanto o paciente do atendimento referenciado?

Essas perguntas devem ser respondidas antes da escolha da estratégia. Após a implementação, o grupo deve validar suas hipóteses com dados de exemplo, observando na prática o comportamento das consultas e das validações.

Por fim, registre no relatório a estratégia escolhida, os critérios que embasaram a decisão e os resultados obtidos nos testes realizados.



REFLEXÃO

O campo `result` do `Exam` exige alguma consideração especial na modelagem?

O campo `result` de um `Exam` é opcional no momento da criação, mas torna-se obrigatório para a emissão do laudo. Isso introduz uma distinção importante entre a modelagem no banco de dados e as regras de negócio.

Do ponto de vista de persistência:

- O banco deve permitir `null` nesse campo, já que o resultado pode não estar disponível no momento da criação.
- Além disso, como o `result` pode conter um texto longo, o tipo de coluna escolhido deve suportar esse volume de dados adequadamente.

Do ponto de vista de regra de negócio, surgem algumas decisões importantes:

- Onde a obrigatoriedade do `result` para emissão de laudo deve ser validada — no `DTO`, no `service` ou em ambos?
- Essa validação deve ocorrer apenas no momento da emissão do laudo, ou também em outras transições de estado?

Também é necessário definir o comportamento ao longo do ciclo de vida do exame:

- O `result` pode ser atualizado após o atendimento ser encerrado?
- Um exame encerrado sem resultado pode ter esse campo preenchido posteriormente sem a necessidade de reabrir o atendimento?

O grupo deve estabelecer regras claras para esses cenários, garantindo consistência entre o modelo de dados e as regras de negócio. Por fim, registre no relatório as decisões tomadas, juntamente com as justificativas técnicas e funcionais.

Procedure → **SimpleProcedure**, **SpecializedProcedure**

A hierarquia de procedimentos é a mais simples em termos de subclasses — apenas dois subtipos — mas a mais rica em termos de ciclo de vida. O `SpecializedProcedure` introduz um fluxo de autorização que não existe em nenhuma outra entidade do sistema.

`SimpleProcedure` carrega apenas `estimatedDuration` — a duração estimada do procedimento em minutos. Sua simplicidade é intencional: procedimentos simples não exigem aprovação, não têm ciclo de vida além de existir ou não existir, e podem ser adicionados a um atendimento em andamento sem restrições além da existência do atendimento.

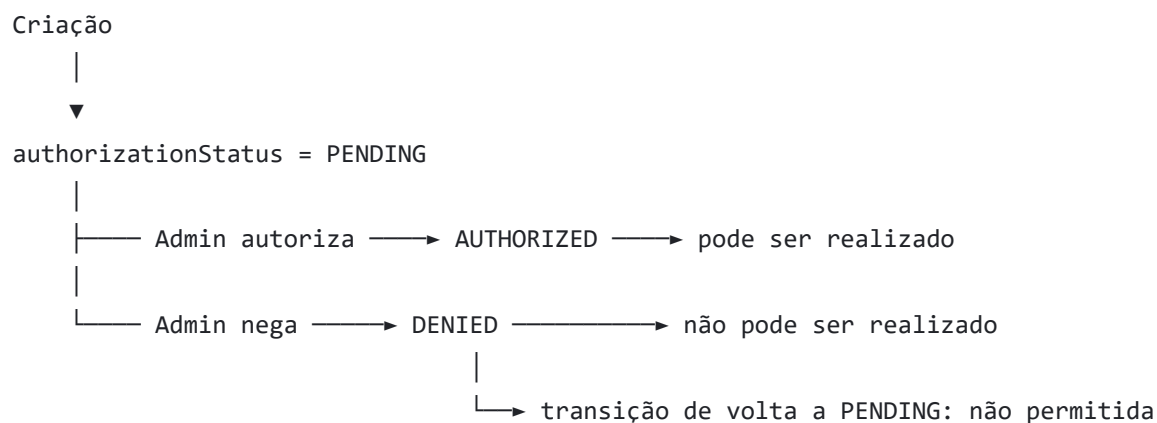
O `SpecializedProcedure` inclui informações específicas necessárias para a execução e controle do procedimento. Entre seus atributos, destacam-se:

- `requiredEquipment`: lista os equipamentos necessários para a realização do procedimento;
- `complexityLevel`: indica o nível de complexidade, classificado como `LOW`, `MEDIUM` ou `HIGH`;
- `requiresAuthorization`: um valor booleano que informa se o procedimento exige aprovação prévia;
- `authorizationStatus`: representa o estado atual da autorização;
- `authorizedAt`: registra o *timestamp* em que a autorização foi concedida, sendo preenchido automaticamente pelo sistema no momento em que um `Admin` realiza a aprovação.

O campo `requiresAuthorization` merece atenção especial: ele é configurado no momento da criação do procedimento e determina se o ciclo de autorização se aplica. Um `SpecializedProcedure` com `requiresAuthorization = false` pode ser realizado imediatamente — como um procedimento simples. Um com `requiresAuthorization = true` deve passar pelo ciclo completo de autorização.

O CICLO DE VIDA DO `SpecializedProcedure`

O ciclo de vida de um procedimento especializado que requer autorização é o fluxo mais complexo desta etapa. O grupo deve compreendê-lo integralmente antes de implementar:



A transição `DENIED → PENDING` é explicitamente bloqueada — uma negação é definitiva para aquele registro de procedimento. Se o médico precisar tentar novamente, deve criar um novo registro de procedimento. Essa decisão tem uma motivação clínica: manter um registro auditável de que um procedimento foi negado, sem possibilidade de sobrescrever essa informação.



REFLEXÃO

O que acontece com um atendimento que tem procedimentos `PENDING` ou `DENIED` ao ser encerrado?

Um atendimento pode ser encerrado com procedimentos especializados ainda em estado `PENDING` ou `DENIED`? Do ponto de vista clínico, encerrar um atendimento com procedimentos não realizados por negação de autorização pode ser um cenário legítimo — o médico documenta o que foi feito e o que não pôde ser feito. Mas o sistema deve simplesmente permitir isso, ou deve alertar — sem bloquear — que há procedimentos não concluídos? O grupo deve definir o comportamento do *endpoint* `PATCH /appointments/{id}/finish` em relação ao estado dos procedimentos associados, implementar a decisão e justificá-la no relatório considerando as implicações clínicas.



REFLEXÃO

O campo `authorizedBy` deve ser um ID ou uma referência completa ao `Admin`?

O campo `authorizedBy` em `SpecializedProcedure` registra quem autorizou o procedimento. Na modelagem, ele é referenciado como o ID do `Admin` autenticado — seguindo o mesmo padrão de `createdBy` e `cancelledBy` em `Schedule`. Mas como esse campo deve ser exposto nas respostas da API? Retornar apenas o ID do administrador pode ser insuficiente para o contexto clínico — saber o nome do administrador que autorizou é mais útil do que seu ID. O grupo deve decidir o que retornar nas respostas dos *endpoints* que expõem procedimentos especializados autorizados — ID, nome, ou ambos — e como essa informação é obtida sem comprometer a arquitetura modular. Registre a decisão no relatório.

PERFIL DE CONSULTAS DA HIERARQUIA `Procedure`

O perfil de consultas de `Procedure` é significativamente diferente do de `Appointment`. Procedimentos são quase sempre consultados no contexto de um atendimento específico — raramente o sistema precisará listar procedimentos de todos os atendimentos misturados. Os casos de consulta mais frequentes são:

- Listar todos os procedimentos de um atendimento específico — com ou sem filtro por tipo.
- Buscar procedimentos especializados com `authorizationStatus = PENDING` para que o `Admin` tome decisões de autorização.
- Agregar procedimentos por tipo e complexidade nos relatórios administrativos.

Esse perfil favorece estratégias que mantêm os dados de cada subtipo bem separados, com bom suporte a filtros por `authorizationStatus` — que só existe em `SpecializedProcedure`.



REFLEXÃO

A hierarquia `Procedure` justifica uma estratégia de herança diferente das anteriores?

Com apenas dois subtipos e um perfil de consultas que raramente mistura os dois, a hierarquia de `Procedure` pode ser o caso mais claro de uma estratégia específica sendo a mais adequada. O grupo deve analisar cada estratégia disponível sob a ótica do perfil de consultas identificado acima. Considere especialmente: como o filtro por `authorizationStatus = PENDING` se comporta em cada estratégia? Esse campo só existe em `SpecializedProcedure` — em uma *Single Table*, ele estaria nulo para todos os `SimpleProcedure`, o que pode complicar as *queries* dos relatórios administrativos. Como cada estratégia lida com esse caso? Registre a análise e a decisão no relatório, e explique se a escolha foi diferente das hierarquias anteriores e por quê.

DECISÃO DOCUMENTADA NO RELATÓRIO

Para cada hierarquia desta etapa, o relatório deve conter uma seção dedicada com a seguinte estrutura mínima:

- **Hierarquia:** nome da hierarquia (`Appointment` ou `Procedure`).
- **Estratégia escolhida:** *Single Table*, *Joined Table* ou *Concrete Table*.
- **Justificativa baseada no domínio:** por que essa estratégia é adequada para o perfil de consultas desta hierarquia específica? Quais padrões de acesso foram considerados?
- **Comparação com escolhas anteriores:** a estratégia é a mesma usada para `User` e `Schedule`? Se sim, por quê? Se não, o que justifica a diferença?
- **Limitações identificadas:** quais são as desvantagens da estratégia escolhida para esta hierarquia? O grupo está ciente delas e as aceita conscientemente?
- **Lições aprendidas:** o que a experiência das etapas anteriores influenciou nesta decisão?



REFLEXÃO

Após implementar as quatro hierarquias do projeto, qual estratégia o grupo recomendaria como padrão para sistemas similares?

Esta é uma reflexão de síntese que pertence ao relatório final. Após implementar herança no banco de dados para `User`, `Schedule`, `Appointment` e `Procedure` — possivelmente com estratégias diferentes para cada uma — o grupo tem uma visão privilegiada das implicações práticas de cada abordagem no TypeORM com SQLite. Se o grupo fosse começar um novo projeto similar amanhã, qual estratégia adotaria como padrão? Em quais situações mudaria de estratégia? Essa reflexão não tem resposta certa — tem respostas bem fundamentadas e respostas superficiais. O grupo deve produzir a primeira.

REQUISITOS FUNCIONAIS

Os requisitos funcionais desta etapa descrevem as operações que completam o ciclo clínico do SGCM — do atendimento ao laudo, passando pelos procedimentos, prontuários e relatórios administrativos. Para cada entidade, são definidos os dados envolvidos, as regras que o sistema deve respeitar e as funcionalidades mínimas esperadas. Os pontos marcados com  não têm resposta única — devem ser discutidos, decididos e justificados no relatório.

Em todos os fluxos, as regras de segurança implementadas na Etapa 2 se aplicam integralmente. Quando uma regra de negócio, de estado ou de segurança impedir a continuação de uma operação, o sistema deve informar o motivo ao cliente por meio de uma resposta de erro no formato RFC 7807 com o status HTTP semanticamente correto, sem alterar nenhum dado no banco de dados.

O grupo deve percorrer cada requisito funcional desta seção e, antes de implementar, responder: quem pode realizar essa operação? Em que estado a entidade envolvida deve estar? Quais outras entidades são verificadas antes da execução? O que muda no banco de dados quando a operação é concluída com sucesso? Essas perguntas estruturam o raciocínio que deve preceder cada implementação — e cujas respostas devem constar no relatório.

1. GESTÃO DE ATENDIMENTOS

O atendimento é a entidade central desta etapa e o elo entre o mundo do agendamento — implementado na Etapa 1 — e o mundo da documentação clínica — prontuários e laudos. Compreender seu ciclo de vida com precisão é o pré-requisito para implementar corretamente tudo que dele depende.

Dados obrigatórios no cadastro (base): `scheduleId`, `type`.

Dados adicionais por tipo:

- **Consultation**: `reason` — obrigatório na criação; `diagnosticHypothesis` — opcional na criação, pode ser atualizado durante o atendimento.
- **Exam**: `examType` — obrigatório na criação; `result` — opcional na criação, pode ser preenchido durante ou após o atendimento, mas é obrigatório para a emissão do laudo.
- **FollowUp**: `originAppointmentId` — obrigatório e validado; `clinicalEvolution` — opcional na criação.

Status e transições:

IN_PROGRESS → FINISHED

A transição é irreversível. Um atendimento encerrado não pode voltar ao estado `IN_PROGRESS` em nenhuma circunstância.

Regras:

- Um atendimento só pode ser criado a partir de um `Schedule` com `status = CONFIRMED`. Qualquer outro status deve ser rejeitado com `400 Bad Request` e uma mensagem que explique o motivo.

- Ao criar o atendimento, o `Schedule` associado deve ter seu status atualizado para `COMPLETED` na mesma operação — de forma atômica. Se qualquer parte dessa operação falhar, nenhuma alteração deve ser persistida.
- O `type` do atendimento não pode ser alterado após a criação — ele é parte da identidade do registro.
- Um `FollowUp` deve referenciar um `Appointment` anterior que pertença ao mesmo `Patient`. Não pode referenciar a si mesmo. Não pode referenciar um atendimento de outro paciente. Essas validações ocorrem no *service* antes da persistência.
- Um atendimento só pode receber `MedicalRecord` após `status = FINISHED`.
- Um `Exam` só pode gerar `Report` após `status = FINISHED` e `result` preenchido.
- O encerramento de um atendimento com `PATCH /appointments/{id}/finish` é a única transição de status — não há um campo `status` editável diretamente via `PUT`.

Funcionalidades mínimas:

- Criar atendimento a partir de agendamento confirmado, informando o tipo e os atributos específicos do subtipo.
- Listar atendimentos com filtros por `doctorId`, `patientId`, `type`, `status` e intervalo de datas, com paginação.
- Buscar atendimento por ID com todos os atributos do subtipo incluídos na resposta.
- Atualizar dados de um atendimento `IN_PROGRESS` — apenas atributos do subtipo, nunca o `type` ou o `scheduleId`.
- Encerrar atendimento (`PATCH /appointments/{id}/finish`).
- Listar atendimentos de um médico (`GET /doctors/{id}/appointments`).
- Listar atendimentos de um paciente (`GET /patients/{id}/appointments`).



REFLEXÃO

O que o *endpoint* `PUT /appointments/{id}` pode atualizar?

Um atendimento possui atributos da base — como `notes`, `startedAt` e `endedAt` — além de atributos específicos de cada subtipo, como `reason`, `diagnosticHypothesis`, `result` e `clinicalEvolution`.

Isso levanta uma questão central: quais desses campos devem ser passíveis de atualização após a criação do atendimento?

Alguns pontos que merecem análise cuidadosa:

- Faz sentido permitir a alteração de `startedAt` em um atendimento já em andamento?
- O `result` de um `Exam` pode ser atualizado após o atendimento ser encerrado, ou apenas enquanto estiver no estado `IN_PROGRESS`?
- Há campos que devem se tornar imutáveis após determinadas transições de estado?

O grupo deve definir, de forma explícita, quais atributos são mutáveis em cada estado do atendimento. A partir dessas definições, é necessário:

- Criar DTOs de atualização que reflitam essas restrições;
- Implementar as validações correspondentes no *service*, garantindo que as regras sejam respeitadas em tempo de execução.

Essas decisões têm implicações clínicas reais e devem ser cuidadosamente justificadas no relatório.



REFLEXÃO

Um `Schedule` pode ter mais de um `Appointment`?

O diagrama de classes estabelece que um `Schedule` pode originar, no máximo, um `Appointment`. Isso levanta algumas questões importantes sobre consistência e regras de negócio. Por exemplo:

- O que deve acontecer se o sistema tentar criar um segundo `Appointment` para o mesmo `scheduleId`?
- O `service` deve verificar previamente se já existe um atendimento associado e, em caso positivo, retornar um `409 Conflict`?

Também é importante definir claramente onde essa verificação será realizada no código, garantindo que a regra seja aplicada de forma consistente. Além disso, considere cenários de ciclo de vida:

- Se um `Appointment` existente for excluído, o `Schedule` pode originar um novo atendimento?
- Faz sentido permitir a exclusão de atendimentos ou seria mais adequado adotar uma estratégia como exclusão lógica (*soft delete*)?

O grupo deve definir essas regras de forma explícita, implementar os mecanismos necessários para garanti-las e manter consistência em todos os pontos do sistema.



REFLEXÃO

Como validar o `originAppointmentId` de um `FollowUp`?

A validação do `originAppointmentId` em `FollowUp` exige uma consulta ao banco para verificar se o atendimento referenciado existe, pertence ao mesmo paciente e não é o próprio atendimento sendo criado. Essa consulta pode ser feita no `service` ou em um `pipe` de validação customizado do NestJS. O grupo deve considerar: onde essa validação é mais coesa — no DTO, no `service`, ou em um `pipe`? Se a validação falhar, qual status HTTP e qual mensagem devem ser retornados? Um `originAppointmentId` que não existe é `404`? Ou é `400` porque o dado fornecido é inválido? Justifique no relatório.

2. GESTÃO DE PROCEDIMENTOS

Procedimentos são realizados durante atendimentos em andamento. Eles enriquecem o registro clínico do atendimento e, no caso dos especializados, introduzem um fluxo de aprovação administrativa que atravessa os perfis de `Doctor` e `Admin`.

Dados obrigatórios (base): `name`, `description`, `appointmentId`, `type`.

Dados adicionais por tipo:

- `SimpleProcedure`: `estimatedDuration` — duração em minutos, deve ser um inteiro positivo.
- `SpecializedProcedure`: `requiredEquipment`, `complexityLevel` (`LOW`, `MEDIUM`, `HIGH`), `requiresAuthorization` — booleano definido no momento da criação.

Regras:

- Procedimentos só podem ser adicionados a atendimentos com `status = IN_PROGRESS`.
- Um `SpecializedProcedure` com `requiresAuthorization = true` é criado com `authorizationStatus = PENDING` automaticamente — esse campo nunca é recebido do cliente na criação.
- Um `SpecializedProcedure` com `requiresAuthorization = false` não passa pelo ciclo de autorização — pode ser considerado como realizado imediatamente.
- A autorização ou negação de um procedimento especializado é exclusiva do `Admin`. O campo `authorizedBy` é preenchido automaticamente com o ID do `Admin` autenticado no momento da autorização — nunca recebido do cliente.
- A transição `DENIED → PENDING` é bloqueada. Uma negação é definitiva para aquele registro.
- Procedimentos não podem ser adicionados a atendimentos `FINISHED`.

Funcionalidades mínimas:

- Adicionar procedimento a atendimento `IN_PROGRESS`, informando o tipo e os atributos específicos.
- Listar procedimentos de um atendimento com filtro por `type` e `authorizationStatus`.
- Buscar procedimento por ID.
- Atualizar dados de um procedimento — com restrições por estado e por tipo.
- Autorizar ou negar procedimento especializado (`PATCH /procedures/{id}/authorization`).
- Excluir procedimento de atendimento `IN_PROGRESS`.



REFLEXÃO

O que o `PATCH /procedures/{id}/authorization` recebe no corpo?

O *endpoint* de autorização deve receber a decisão do `Admin` — autorizar ou negar um procedimento. A forma como essa decisão é representada no corpo da requisição impacta diretamente a clareza e a consistência da API. Algumas abordagens possíveis incluem:

- Um campo `action` com valores como `AUTHORIZE` ou `DENY`;
- Um campo `authorizationStatus` utilizando diretamente os valores de um enum;
- Uma abordagem mais semântica, com dois *endpoints* distintos:
 - `PATCH /procedures/{id}/authorize`
 - `PATCH /procedures/{id}/deny`

Cada alternativa traz implicações diferentes em termos de:

- Clareza da API;
- Simplicidade ou complexidade na validação dos DTOs;
- Qualidade da documentação gerada no Swagger.

O grupo deve escolher uma dessas abordagens, aplicá-la de forma consistente em toda a API e justificar a decisão no relatório. Além disso, considere um aspecto importante de usabilidade e comunicação: *Ao negar um procedimento, o `Admin` deve ser obrigado (ou ao menos incentivado) a informar um motivo?*

Esse tipo de informação pode ser valioso para que o médico compreenda a decisão e tome as ações necessárias posteriormente.



REFLEXÃO

Um procedimento pode ser atualizado após o atendimento ser encerrado?

A regra define que procedimentos só podem ser adicionados a atendimentos `IN_PROGRESS`. Mas e a atualização de um procedimento já existente — ela também é restrita ao estado `IN_PROGRESS` do atendimento? Ou um `Admin` poderia atualizar dados de um procedimento em um atendimento já encerrado para fins de correção? O grupo deve definir essa política considerando as implicações clínicas — alterar o registro de um procedimento após o atendimento ser encerrado e o prontuário gerado pode criar inconsistências entre os dois documentos. A decisão deve ser justificada no relatório.

3. PRONTUÁRIOS

O prontuário é o documento clínico que o médico produz ao encerrar um atendimento. Ele não tem ciclo de vida complexo — existe ou não existe — mas tem regras de acesso e de imutabilidade que o tornam um dos registros mais sensíveis do sistema.

Dados obrigatórios: `diagnosis`, `appointmentId`.

Dados opcionais: `prescription`, `notes`.

Regras:

- Um prontuário só pode ser criado para um `Appointment` com `status = FINISHED`.
- Cada atendimento pode ter no máximo um prontuário — tentativas de criar um segundo devem retornar `409 Conflict`.
- Um prontuário não pode ser excluído — representa um documento clínico permanente. Qualquer tentativa de exclusão deve retornar `409 Conflict` com uma mensagem explicativa.
- Atualizações são permitidas, mas o campo `lastUpdatedBy` deve ser preenchido automaticamente com o ID do usuário autenticado que realizou a atualização — nunca recebido do cliente.
- O acesso ao prontuário é controlado por perfil e por recurso: `Admin` acessa qualquer prontuário; `Doctor` acessa apenas prontuários de atendimentos que realizou; `Patient` acessa apenas seus próprios prontuários.

Funcionalidades mínimas:

- Criar prontuário para atendimento encerrado.
- Buscar prontuário de um atendimento específico.
- Atualizar prontuário — com rastreamento de quem atualizou.
- Listar prontuários de um paciente (`GET /patients/{id}/records`).
- Listar prontuários de atendimentos de um médico (`GET /doctors/{id}/records`).

REFLEXÃO



O prontuário deve ser criado automaticamente ao encerrar o atendimento ou manualmente pelo médico?

Há duas abordagens para a criação do prontuário. Na primeira, o encerramento do atendimento (`PATCH /appointments/{id}/finish`) cria automaticamente um prontuário vazio — que o médico preenche posteriormente via `PUT /records/{id}`. Na segunda, o prontuário é criado explicitamente pelo médico após o encerramento, em uma operação separada. A primeira abordagem garante que todo atendimento encerrado tem um prontuário; a segunda dá mais controle ao médico mas cria o risco de atendimentos encerrados sem prontuário. Qual abordagem faz mais sentido para o contexto clínico do SGCM? O grupo deve avaliar os *trade-offs* e justificar a decisão no relatório. Se optar pela criação automática, como isso afeta a atomicidade do encerramento do atendimento?



REFLEXÃO

Quais campos do prontuário podem ser atualizados e por quanto tempo?

O prontuário não pode ser excluído, mas pode ser atualizado. Essa permissão é irrestrita — um prontuário pode ser atualizado a qualquer momento, indefinidamente? Ou faz sentido restringir atualizações a um período após o encerramento do atendimento — por exemplo, apenas nas 48 horas seguintes? E todos os campos podem ser alterados, ou apenas `prescription` e `notes`, preservando o `diagnosis` original? O grupo deve definir essas regras considerando as implicações clínicas e legais de alterar documentos médicos, e implementá-las de forma consistente. A decisão deve ser justificada no relatório.



REFLEXÃO

Como o campo `lastUpdatedBy` interage com o campo `createdBy`?

O prontuário tem pelo menos dois campos de rastreabilidade: quem o criou — implicitamente o médico do atendimento — e quem o atualizou pela última vez — `lastUpdatedBy`. Mas o grupo deve decidir se o `createdBy` deve ser explicitamente armazenado no prontuário ou se é suficiente derivá-lo do médico do atendimento associado. Se `createdBy` for explicitamente armazenado, ele deve ser preenchido automaticamente com o usuário autenticado no momento da criação. E se o médico que criou e o que atualizou pela última vez forem a mesma pessoa — `lastUpdatedBy` deve ser preenchido mesmo assim? O grupo deve definir essas regras de rastreabilidade e justificá-las no relatório.

4. LAUDOS

O laudo é o documento mais formal do sistema — gerado para exames, assinado digitalmente pelo código de validação e verificável publicamente. Sua implementação combina regras de negócio rigorosas, geração de arquivo binário e um *endpoint* sem autenticação — o único do sistema além dos de *login* e *refresh*.

Regras de emissão:

- Um laudo só pode ser emitido para um `Appointment` do tipo `Exam`, com `status = FINISHED` e `result` preenchido.
- O `validationCode` é um UUID gerado automaticamente pelo sistema no momento da emissão — nunca fornecido pelo cliente, nunca nulo.
- Cada exame pode ter múltiplos laudos ao longo do tempo, mas apenas um com `status = ACTIVE` simultaneamente. Tentar emitir um novo laudo para um exame que já tem laudo `ACTIVE` deve retornar `409 Conflict`.
- O laudo deve ser disponibilizado em PDF com as informações obrigatórias — nome do paciente, nome do médico, tipo de exame, resultado, data de emissão e `validationCode` em destaque.

Regras de revogação:

- Apenas o `Admin` ou o `Doctor` que emitiu o laudo podem revogá-lo. O controle por recurso deve ser implementado no *service*.
- A revogação exige `revokedReason` — obrigatório no corpo da requisição.
- Os campos `revokedAt` e `revokedBy` são preenchidos automaticamente pelo sistema — `revokedAt` com o *timestamp* atual e `revokedBy` com o ID do usuário autenticado.
- A transição `ACTIVE → REVOKED` é irreversível.
- Após a revogação, um novo laudo pode ser emitido para o mesmo exame — o sistema permite múltiplos laudos históricos, desde que apenas um esteja `ACTIVE`.

Regras de validação pública:

- O *endpoint* `GET /reports/validate/{code}` é o único *endpoint* do sistema sem autenticação além dos de autenticação da Etapa 2.
- Deve ser marcado com `@Public()` — o decorator implementado na Etapa 2.
- Retorna dados básicos do laudo para confirmar autenticidade — sem expor informações clínicas sensíveis além do necessário para a validação.
- Para laudos com `status = REVOKED`, deve retornar os dados básicos e indicar claramente que o laudo foi revogado — sem esconder a existência do registro.

Funcionalidades mínimas:

- Emitir laudo para exame encerrado com resultado (`POST /appointments/{id}/report`).
- Baixar laudo em PDF (`GET /reports/{id}/pdf`).
- Validar laudo por código — público (`GET /reports/validate/{code}`).
- Revogar laudo (`PATCH /reports/{id}/revoke`).
- Listar laudos de um paciente (`GET /patients/{id}/reports`).
- Listar laudos emitidos por um médico (`GET /doctors/{id}/reports`).



REFLEXÃO

O que o *endpoint* de validação pública deve retornar para um laudo revogado?

Quando um laudo é revogado, ele continua armazenado no banco de dados — a revogação não implica sua exclusão. Nesse contexto, o *endpoint* de validação pública, que recebe o `validationCode`, precisa lidar com diferentes estados do laudo.

Para um laudo com status `ACTIVE`, o comportamento é direto: o sistema retorna as informações normalmente. Já para um laudo `REVOKED`, surgem algumas possibilidades de design:

- Retornar os dados básicos do laudo, indicando explicitamente que ele foi revogado;
- Retornar apenas a informação de que o laudo foi revogado, sem expor seus dados;
- Retornar `404 Not Found`, ocultando completamente a existência do laudo.

Cada uma dessas abordagens tem implicações importantes para os consumidores externos da API. Por exemplo, uma seguradora que utiliza o código de validação precisa, no mínimo, saber se o laudo é válido ou se foi revogado. Em alguns casos, pode ser relevante também entender o motivo da revogação.

Diante disso, o grupo deve definir de forma explícita o contrato de resposta para cada estado possível do laudo, equilibrando:

- Transparência para quem consome o serviço;
- Privacidade e proteção de dados sensíveis.

Por fim, registre no relatório a decisão adotada e a justificativa por trás dela.



REFLEXÃO

O PDF do laudo deve conter o QR code do link de validação?

Um laudo em papel costuma ter um mecanismo de verificação de autenticidade — como um QR code que aponta para o *endpoint* de validação pública. O grupo deve refletir: faz sentido incluir o `validationCode` apenas como texto no PDF, ou seria mais útil incluir também um QR code que aponte para `GET /reports/validate/{code}`? A segunda opção exige uma biblioteca adicional para geração de QR code — como `qrcode` — mas melhora significativamente a usabilidade do documento. A decisão deve considerar o esforço de implementação e o valor agregado ao usuário, e ser justificada no relatório.



REFLEXÃO

Como controlar o acesso ao PDF do laudo?

O *endpoint* `GET /reports/{id}/pdf` requer autenticação — apenas `Admin`, `Doctor` e `Patient` autorizados podem acessá-lo. Mas o controle por recurso aqui é mais complexo do que nos demais *endpoints*: um `Patient` só pode baixar o PDF de seus próprios laudos; um `Doctor` só pode baixar o PDF de laudos que emitiu. Como o *service* verificará essas condições? E se o laudo estiver `REVOKED` — um `Patient` ainda pode baixar o PDF do laudo revogado? O grupo deve definir essas regras e implementá-las de forma consistente no *service*, justificando cada decisão no relatório.

5. RELATÓRIOS ADMINISTRATIVOS

Os relatórios administrativos oferecem ao `Admin` uma visão consolidada e quantitativa do funcionamento da clínica. Eles são operações de leitura puras — não criam, não atualizam e não removem dados. Sua complexidade está na lógica de agregação e na abrangência dos filtros que devem suportar.

Regras gerais:

- Todos os relatórios são exclusivos do `Admin` — retornam `403 Forbidden` para qualquer outro perfil autenticado.
- Os dados são consultados em tempo real — sem cache.
- Todos os relatórios suportam filtro por intervalo de datas via parâmetros `startDate` e `endDate`.
- Intervalos de datas inválidos — `startDate` posterior a `endDate` — devem retornar `400 Bad Request`.

Relatório de agendamentos (`GET /admin/reports/schedules`):

Consolida informações sobre os agendamentos da clínica. Deve retornar:

- Total de agendamentos por `status` (`PENDING`, `CONFIRMED`, `CANCELLED`, `COMPLETED`).
- Total de agendamentos por `type` (`IN_PERSON`, `ONLINE`, `HOME`).
- Total geral no período filtrado.

Relatório de atendimentos (`GET /admin/reports/appointments`):

Consolida informações sobre os atendimentos realizados. Deve retornar:

- Total de atendimentos por `type` (`CONSULTATION`, `EXAM`, `FOLLOW_UP`).
- Total de atendimentos por `status` (`IN_PROGRESS`, `FINISHED`).
- Suporte a filtro adicional por `doctorId`.

Relatório de procedimentos (`GET /admin/reports/procedures`):

Consolida informações sobre os procedimentos registrados. Deve retornar:

- Total de procedimentos por `type` (`SIMPLE`, `SPECIALIZED`).
- Total de procedimentos especializados por `authorizationStatus` (`PENDING`, `AUTHORIZED`, `DENIED`).

- Total de procedimentos especializados por `complexityLevel` (`LOW`, `MEDIUM`, `HIGH`).

Relatório de ocupação de agenda (`GET /admin/reports/doctors/{id}/occupation`):

Consolida a ocupação da agenda de um médico específico em um período. Deve retornar:

- Total de agendamentos no período.
- Total por `status`.
- Taxa de ocupação — definida e calculada pelo grupo.



REFLEXÃO

Como estruturar a resposta dos relatórios?

Os relatórios retornam dados agregados, e não listas de registros. Isso implica um formato de resposta diferente dos *endpoints* de listagem: não há paginação nem arrays de entidades individuais. Diante disso, o grupo deve definir uma estrutura de resposta clara e consistente para cada relatório. Um ponto central dessa definição é a representação de totais por categoria. Algumas opções incluem:

- Um objeto com categorias como chaves, por exemplo: `{ "PENDING": 12, "CONFIRMED": 34 }`
- Um array de objetos, por exemplo: `[{ "status": "PENDING", "count": 12 }]`

Cada abordagem tem suas vantagens:

- A estrutura em objeto é mais compacta e direta;
- O array de objetos é mais flexível e extensível, especialmente para evoluções futuras.

O grupo deve escolher um formato, aplicá-lo de forma consistente em todos os relatórios e documentá-lo adequadamente no Swagger, incluindo exemplos realistas. Além disso, considere a integração com a infraestrutura existente:

- O *Transform Interceptor*, definido na Etapa 2, também se aplica a esses *endpoints*?
- Como a resposta dos relatórios será encapsulada no envelope padrão `{ data, meta }`?

Essas decisões devem ser explicitadas e justificadas no relatório, garantindo consistência em toda a API.



REFLEXÃO

Como definir e calcular a taxa de ocupação de um médico?

A taxa de ocupação é uma métrica que o grupo deve definir com precisão antes de implementar — uma métrica mal definida produz números que não fazem sentido. O que entra no cálculo? O denominador é o total de agendamentos criados no período, ou o total de slots disponíveis — que o sistema atual não controla? Se for o total de agendamentos, agendamentos cancelados entram no cálculo? Uma taxa calculada como $(\text{CONFIRMED} + \text{COMPLETED}) / (\text{PENDING} + \text{CONFIRMED} + \text{CANCELLED} + \text{COMPLETED})$ tem um significado diferente de $\text{COMPLETED} / (\text{CONFIRMED} + \text{COMPLETED})$. O grupo deve escolher uma fórmula, documentá-la claramente no Swagger — para que o Admin entenda o que o número significa — e justificar a escolha no relatório. Uma métrica documentada com clareza é mais valiosa do que uma métrica "certa" sem explicação.



REFLEXÃO

Os relatórios devem ser implementados com *queries* SQL otimizadas ou com lógica em memória?

Há duas abordagens para implementar a agregação dos relatórios. A primeira é usar *queries* SQL com `GROUP BY` e funções de agregação — `COUNT`, `SUM` — que delegam o processamento ao banco de dados, o que é mais eficiente para volumes grandes de dados. A segunda é buscar os registros brutos e agregar em memória no *service* — mais simples de implementar, mas menos eficiente. O TypeORM suporta ambas as abordagens — *queries* nativas com `createQueryBuilder` e processamento em JavaScript. Para o volume de dados esperado em um projeto didático com SQLite, ambas funcionam. Mas o grupo deve fazer a escolha conscientemente, considerando o que seria adequado em um sistema real com milhares de registros, e justificar no relatório.

ENDPOINTS OBRIGATÓRIOS

Os *endpoints* listados abaixo são obrigatórios e devem ser implementados exatamente conforme especificado — mesmo método HTTP, mesma rota, mesmos códigos de status. Todos devem estar protegidos com os *guards* implementados na Etapa 2, exceto o *endpoint* explicitamente marcado como público.

O grupo pode implementar *endpoints* adicionais que façam sentido para o domínio clínico. Todo *endpoint* adicional deve seguir as mesmas convenções de autenticação, autorização, paginação e tratamento de erros estabelecidas ao longo do projeto, estar documentado no Swagger e registrado no relatório com a justificativa de sua implementação.

SOBRE A TABELA DE CONTROLE DE ACESSO

Cada tabela de *endpoints* desta seção inclui as colunas "Auth" e "Perfis", que indicam o requisito mínimo de autenticação e os perfis autorizados para cada *endpoint*. A coluna "Controle por recurso" indica quando a verificação de propriedade do recurso deve ocorrer no *service* — além da verificação de perfil no *guard*.

O grupo deve expandir a tabela de controle de acesso iniciada na Etapa 2 para cobrir todos os novos *endpoints* desta etapa. Essa tabela consolidada — cobrindo as três etapas — deve constar no relatório final.

ATENDIMENTOS

MÉTODO	ENDPOINT	DESCRIÇÃO	AUTH	PERFIS	CONTROLE POR RECURSO	STATUS HTTP
POST	<code>/appointments</code>	Criar atendimento a partir de agendamento confirmado, informando o tipo e atributos do subtipo.	JWT + Roles	Admin, Doctor	—	201, 400, 404, 409
GET	<code>/appointments</code>	Listar atendimentos com filtros por <code>doctorId</code> , <code>patientId</code> , <code>type</code> , <code>status</code> e intervalo de datas.	JWT + Roles	Admin	—	200
GET	<code>/appointments/{id}</code>	Buscar atendimento por ID com todos os atributos do subtipo.	JWT + Roles	Admin, Doctor, Patient	Doctor/ Patient: próprios	200, 403, 404

MÉTODO	ENDPOINT	DESCRIÇÃO	AUTH	PERFIS	CONTROLE POR RECURSO	STATUS HTTP
PUT	<code>/appointments/{id}</code>	Atualizar atributos mutáveis de um atendimento <code>IN_PROGRESS</code> .	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 400, 403, 404
PATCH	<code>/appointments/{id}/finish</code>	Encerrar atendimento — transição irreversível para <code>FINISHED</code> .	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 400, 403, 404
GET	<code>/doctors/{id}/appointments</code>	Listar atendimentos realizados por um médico, com filtros e paginação.	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 403, 404
GET	<code>/patients/{id}/appointments</code>	Listar atendimentos de um paciente, com filtros e paginação.	JWT + Roles	Admin, Patient	Patient: próprios	200, 403, 404

Regras de negócio nos códigos de status:

- `POST /appointments` retorna `409` quando já existe um `Appointment` para o `scheduleId` informado.
- `POST /appointments` retorna `400` quando o `Schedule` não está com status `CONFIRMED`.
- `PATCH /appointments/{id}/finish` retorna `400` quando o atendimento já está `FINISHED`.
- `PUT /appointments/{id}` retorna `400` quando o atendimento está `FINISHED` — atributos não podem ser alterados após o encerramento, exceto conforme definido pelo grupo.

Parâmetros de *query* para listagens de atendimentos:

```
GET /appointments?doctorId=3&type=EXAM&status=FINISHED&startDate=2025-09-01&endDate=2025-09-01
GET /doctors/3/appointments?type=CONSULTATION&status=IN_PROGRESS
GET /patients/7/appointments?type=EXAM
```

PROCEDIMENTOS

MÉTODO	ENDPOINT	DESCRIÇÃO	AUTH	PERFIS	CONTROLE POR RECURSO	STATUS HTTP
--------	----------	-----------	------	--------	----------------------	-------------

MÉTODO	ENDPOINT	DESCRIÇÃO	AUTH	PERFIS	CONTROLE POR RECURSO	STATUS HTTP
POST	<code>/appointments/{id}/procedures</code>	Adicionar procedimento a atendimento <code>IN_PROGRESS</code> .	JWT + Roles	Admin, Doctor	Doctor: atendimentos próprios	201, 400, 403, 404
GET	<code>/appointments/{id}/procedures</code>	Listar procedimentos de um atendimento com filtro por <code>type</code> e <code>authorizationStatus</code> .	JWT + Roles	Admin, Doctor, Patient	Doctor/ Patient: próprios	200, 403, 404
GET	<code>/procedures/{id}</code>	Buscar procedimento por ID com todos os atributos do subtipo.	JWT + Roles	Admin, Doctor, Patient	Doctor/ Patient: próprios	200, 403, 404
PUT	<code>/procedures/{id}</code>	Atualizar dados de um procedimento — com restrições por estado e tipo.	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 400, 403, 404
PATCH	<code>/procedures/{id}/authorization</code>	Autorizar ou negar procedimento especializado.	JWT + Roles	Admin	—	200, 400, 403, 404
DELETE	<code>/procedures/{id}</code>	Excluir procedimento de atendimento <code>IN_PROGRESS</code> .	JWT + Roles	Admin, Doctor	Doctor: próprios	204, 403, 404, 409

Regras de negócio nos códigos de status:

- `POST /appointments/{id}/procedures` retorna `400` quando o atendimento está `FINISHED`.
- `PATCH /procedures/{id}/authorization` retorna `400` quando o procedimento não é do tipo `SPECIALIZED`, quando `requiresAuthorization = false`, ou quando `authorizationStatus = DENIED` — a transição de volta a `PENDING` não é permitida.
- `DELETE /procedures/{id}` retorna `409` quando o atendimento associado está `FINISHED`.

Corpo esperado para `PATCH /procedures/{id}/authorization`:

O grupo deve definir a estrutura deste corpo — a especificação propositalmente não a prescreve. A decisão deve ser justificada no relatório e documentada no Swagger com exemplos para os casos de autorização e negação.

PRONTUÁRIOS

MÉTODO	ENDPOINT	DESCRIÇÃO	AUTH	PERFIS	CONTROLE	STATUS
--------	----------	-----------	------	--------	----------	--------

					POR RECURSO	HTTP
POST	<code>/appointments/{id}/records</code>	Criar prontuário para atendimento <code>FINISHED</code> .	JWT + Roles	Admin, Doctor	Doctor: atendimentos próprios	201, 400, 403, 404, 409
GET	<code>/appointments/{id}/records</code>	Buscar prontuário de um atendimento.	JWT + Roles	Admin, Doctor, Patient	Doctor/Patient: próprios	200, 403, 404
PUT	<code>/records/{id}</code>	Atualizar prontuário — com rastreamento de <code>lastUpdatedBy</code> .	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 400, 403, 404
GET	<code>/patients/{id}/records</code>	Listar prontuários de um paciente com paginação.	JWT + Roles	Admin, Doctor, Patient	Doctor/Patient: próprios	200, 403, 404
GET	<code>/doctors/{id}/records</code>	Listar prontuários de atendimentos realizados por um médico.	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 403, 404

Regras de negócio nos códigos de status:

- `POST /appointments/{id}/records` retorna `400` quando o atendimento está `IN_PROGRESS`.
- `POST /appointments/{id}/records` retorna `409` quando já existe um prontuário para aquele atendimento.
- Qualquer tentativa de `DELETE` em um prontuário deve retornar `409` — prontuários não podem ser excluídos.



REFLEXÃO

O *endpoint* `DELETE /records/{id}` deve existir?

A especificação determina que prontuários não podem ser excluídos. Isso significa que o *endpoint* `DELETE /records/{id}` não deve existir — ou deve existir e retornar `409 Conflict` com uma mensagem explicativa? Há uma diferença importante entre as duas abordagens: a primeira torna a restrição invisível na documentação do Swagger — o cliente simplesmente não encontra um *endpoint* de exclusão; a segunda torna a restrição explícita — o cliente sabe que tentou excluir e recebeu uma negativa com motivo. O grupo deve decidir qual abordagem adotar, considerando a clareza da API para quem a consome, e justificar no relatório.

LAUDOS

MÉTODO	ENDPOINT	DESCRIÇÃO	AUTH	PERFIS	CONTROLE POR RECURSO	STATUS HTTP
POST	<code>/appointments/{id}/report</code>	Emitir laudo para exame <code>FINISHED</code> com <code>result</code> preenchido.	JWT + Roles	Admin, Doctor	Doctor: atendimentos próprios	201, 400, 403, 404, 409
GET	<code>/reports/{id}/pdf</code>	Baixar laudo em PDF.	JWT + Roles	Admin, Doctor, Patient	Doctor/ Patient: próprios	200, 403, 404
GET	<code>/reports/validate/{code}</code>	Validar laudo por código de validação — endpoint público.	Público	Qualquer um	—	200, 404
PATCH	<code>/reports/{id}/revoke</code>	Revogar laudo — requer <code>revokedReason</code> no corpo.	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 400, 403, 404
GET	<code>/patients/{id}/reports</code>	Listar laudos de um paciente com paginação.	JWT + Roles	Admin, Doctor, Patient	Doctor/ Patient: próprios	200, 403, 404
GET	<code>/doctors/{id}/reports</code>	Listar laudos emitidos por um médico com paginação.	JWT + Roles	Admin, Doctor	Doctor: próprios	200, 403, 404

Regras de negócio nos códigos de status:

- `POST /appointments/{id}/report` retorna `400` quando o atendimento não é do tipo `EXAM`, quando `status != FINISHED`, ou quando `result` está vazio ou nulo.
- `POST /appointments/{id}/report` retorna `409` quando já existe um laudo `ACTIVE` para aquele exame.
- `PATCH /reports/{id}/revoke` retorna `400` quando o laudo já está `REVOKED`.
- `PATCH /reports/{id}/revoke` retorna `400` quando `revokedReason` está ausente ou vazio.
- `GET /reports/validate/{code}` retorna `404` quando nenhum laudo com aquele código existe — independentemente do status do laudo encontrado.
- `GET /reports/{id}/pdf` retorna a resposta com `Content-Type: application/pdf` e `Content-Disposition: attachment; filename="laudo-{validationCode}.pdf"`.

Corpo esperado para `PATCH /reports/{id}/revoke` :

```
{
  "revokedReason": "Resultado incorreto identificado após nova análise."
}
```

Atenção especial — `GET /reports/validate/{code}`:

Este é o único *endpoint* desta etapa sem autenticação. Ele deve ser marcado com `@Public()` e não deve ter `@ApiBearerAuth()` na documentação. O grupo deve verificar que o `JwtAuthGuard` global o ignora corretamente — um erro de configuração que force autenticação neste *endpoint* compromete o propósito da validação pública.

**REFLEXÃO**

A rota `GET /reports/validate/{code}` pode entrar em conflito com `GET /reports/{id}`?

O roteamento do NestJS processa as rotas na ordem em que são declaradas no *controller*. Se `GET /reports/{id}` for declarado antes de `GET /reports/validate/{code}`, o NestJS pode interpretar `validate` como um valor para o parâmetro `{id}` — causando um comportamento inesperado. O grupo deve investigar como o NestJS resolve esse tipo de ambiguidade de rota e garantir que as rotas sejam declaradas na ordem correta. Esse é um problema clássico de roteamento em APIs REST que o grupo deve resolver conscientemente e documentar no relatório.

RELATÓRIOS ADMINISTRATIVOS

MÉTODO	ENDPOINT	DESCRIÇÃO	AUTH	PERFIS	STATUS HTTP
GET	<code>/admin/reports/schedules</code>	Totais de agendamentos por status e modalidade no período.	JWT + Roles	Admin	200, 400, 403
GET	<code>/admin/reports/appointments</code>	Totais de atendimentos por tipo e status no período.	JWT + Roles	Admin	200, 400, 403
GET	<code>/admin/reports/procedures</code>	Totais de procedimentos por tipo, autorização e complexidade.	JWT + Roles	Admin	200, 400, 403
GET	<code>/admin/reports/doctors/{id}/occupation</code>	Taxa de ocupação da agenda de um médico no período.	JWT + Roles	Admin	200, 400, 403, 404

Parâmetros de *query* para relatórios:

Todos os relatórios suportam os seguintes parâmetros. Os parâmetros de paginação **não se aplicam** — relatórios retornam dados agregados, não listas paginadas.

PARÂMETRO	TIPO	OBRIGATÓRIO	DESCRIÇÃO
<code>startDate</code>	<code>string</code> (ISO 8601)	Não	Início do período de análise.
<code>endDate</code>	<code>string</code> (ISO 8601)	Não	Fim do período de análise.

`startDate` posterior a `endDate` retorna `400 Bad Request`. Quando não informados, o relatório considera todos os registros sem filtro de data.

Exemplos de uso:

```
GET /admin/reports/schedules?startDate=2025-09-01&endDate=2025-09-30
GET /admin/reports/appointments?startDate=2025-09-01&doctorId=3
GET /admin/reports/doctors/3/occupation?startDate=2025-09-01&endDate=2025-09-30
```

CONTROLE DE ACESSO POR RECURSO — SÍNTESE DESTA ETAPA

A tabela abaixo sintetiza os *endpoints* desta etapa que exigem controle por recurso no *service*, além do controle por perfil no *guard*. O grupo deve implementar cada verificação no *service* correspondente — nunca no *controller*.

ENDPOINT	DOCTOR	PATIENT	ADMIN
<code>GET /appointments/{id}</code>	Apenas atendimentos que realizou	Apenas os próprios	Irrestrito
<code>PUT /appointments/{id}</code>	Apenas atendimentos que realizou	—	Irrestrito
<code>PATCH /appointments/{id}/finish</code>	Apenas atendimentos que realizou	—	Irrestrito
<code>GET /doctors/{id}/appointments</code>	Apenas a própria agenda	—	Irrestrito
<code>GET /patients/{id}/appointments</code>	—	Apenas os próprios	Irrestrito
<code>POST /appointments/{id}/procedures</code>	Apenas em atendimentos que realizou	—	Irrestrito
<code>GET /appointments/{id}/procedures</code>	Apenas de atendimentos que realizou	Apenas dos próprios	Irrestrito

ENDPOINT	DOCTOR	PATIENT	ADMIN
GET /procedures/{id}	Apenas de atendimentos que realizou	Apenas dos próprios	Irrestrito
PUT /procedures/{id}	Apenas de atendimentos que realizou	—	Irrestrito
DELETE /procedures/{id}	Apenas de atendimentos que realizou	—	Irrestrito
POST /appointments/{id}/records	Apenas em atendimentos que realizou	—	Irrestrito
GET /appointments/{id}/records	Apenas de atendimentos que realizou	Apenas os próprios	Irrestrito
PUT /records/{id}	Apenas de atendimentos que realizou	—	Irrestrito
GET /patients/{id}/records	Apenas de pacientes que atendeu	Apenas os próprios	Irrestrito
GET /doctors/{id}/records	Apenas a própria lista	—	Irrestrito
POST /appointments/{id}/report	Apenas em atendimentos que realizou	—	Irrestrito
GET /reports/{id}/pdf	Apenas laudos que emituiu	Apenas os próprios	Irrestrito
PATCH /reports/{id}/revoke	Apenas laudos que emituiu	—	Irrestrito
GET /patients/{id}/reports	Apenas de pacientes que atendeu	Apenas os próprios	Irrestrito
GET /doctors/{id}/reports	Apenas a própria lista	—	Irrestrito

**REFLEXÃO****A tabela anterior está completa e correta para o domínio clínico?**

Analise cada linha da tabela com senso crítico. Verifique se há situações em que a política de acesso definida pode estar excessivamente restritiva, a ponto de impedir operações que seriam legítimas em um contexto clínico real. Por exemplo:

- Um **Doctor** que substitui temporariamente outro médico deveria ter acesso aos atendimentos desse colega?
- Um **Patient** que deseja compartilhar seu prontuário com outro médico deveria ter algum mecanismo de suporte a esse fluxo?

Esses cenários podem não estar no escopo atual do projeto, mas devem ser reconhecidos como limitações do modelo de controle de acesso adotado. O grupo deve documentar essas restrições de forma explícita no relatório. Além disso, avalie o inverso: *Há casos em que a política atual pode estar permissiva demais?*

O grupo deve revisar a tabela de forma sistemática, propor ajustes quando necessário e justificar cada alteração no relatório, garantindo coerência entre segurança, usabilidade e simplicidade do modelo.

PARÂMETROS DE QUERY PADRÃO

Os parâmetros de paginação definidos na Etapa 1 continuam válidos para todos os *endpoints* de listagem desta etapa. O envelope `{ data, meta }` com os campos de paginação e os metadados do *Transform Interceptor* deve ser aplicado de forma consistente.

Os relatórios administrativos são a exceção — eles retornam dados agregados, não listas paginadas, e portanto não suportam `page`, `limit` ou `sort`. O grupo deve garantir que o *Transform Interceptor* envolva corretamente as respostas dos relatórios — sem tentar adicionar campos de paginação que não fazem sentido para esse tipo de resposta.



REFLEXÃO

Endpoints adicionais desta etapa

Os *endpoints* acima cobrem o mínimo esperado. O grupo deve refletir sobre o domínio clínico e identificar se há *endpoints* adicionais que agregariam valor real ao sistema — por exemplo, um *endpoint* para listar todos os exames de um paciente que ainda não têm laudo emitido, ou um *endpoint* para buscar atendimentos com procedimentos pendentes de autorização. Todo *endpoint* adicional deve seguir as mesmas convenções estabelecidas ao longo do projeto, estar documentado no Swagger com exemplos realistas e ser registrado no relatório com a justificativa de sua implementação e como se encaixa na arquitetura.

REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais desta etapa definem **como** as funcionalidades clínicas devem ser construídas — com que qualidade, com que segurança e com que cuidado em relação à integridade dos dados. Esta é a etapa que introduz os registros mais sensíveis do sistema — prontuários e laudos — e essa sensibilidade deve se refletir em cada decisão de implementação.

Os requisitos gerais definidos no documento base e os requisitos específicos das Etapas 1 e 2 continuam válidos e devem ser respeitados. Esta seção complementa esses requisitos com as especificidades desta etapa final.

1. INTEGRIDADE DOS DADOS CLÍNICOS

Prontuários e laudos são documentos médicos com implicações que vão além do técnico. Em um sistema real, a adulteração ou exclusão desses registros poderia ter consequências legais e éticas graves. Para o escopo deste projeto, o grupo deve implementar proteções que reflitam essa realidade — mesmo que de forma simplificada.

REGISTROS IMUTÁVEIS EM TERMOS DE EXCLUSÃO

Os seguintes registros não podem ser excluídos em nenhuma circunstância — nem por `Admin`, nem por qualquer outro perfil:

- `Appointment` — representa um atendimento clínico que ocorreu.
- `MedicalRecord` — representa o registro clínico oficial de um atendimento.
- `Report` — representa um laudo emitido, mesmo que revogado.

Qualquer tentativa de exclusão desses registros deve retornar `405 Method Not Allowed` — indicando que aquela operação simplesmente não existe para aquele recurso — ou `409 Conflict` com uma mensagem explicativa. O grupo deve escolher o status mais adequado semanticamente e justificar no relatório.

ATENDIMENTOS ENCERRADOS

Um atendimento com `status = FINISHED` representa um encontro clínico concluído. Além de não poder ser excluído, suas transições de estado são irreversíveis. O grupo deve garantir que o sistema rejeite qualquer tentativa de reverter o status de um atendimento encerrado — seja via `PUT` tentando alterar o campo `status` diretamente, seja via qualquer outro mecanismo.

**REFLEXÃO**

Como o sistema deve se comportar quando uma operação proibida é tentada sobre um registro protegido?

Há uma diferença semântica importante entre `405 Method Not Allowed` e `409 Conflict`. O primeiro indica que aquele método HTTP simplesmente não é suportado para aquele recurso — não existe um *handler* para `DELETE /records/{id}`. O segundo indica que a operação existe mas não pode ser executada no estado atual do recurso. Para registros que nunca podem ser excluídos — como prontuários — qual status comunica melhor a restrição ao consumidor da API? Um desenvolvedor que recebe `405` entende que exclusão nunca é possível; um que recebe `409` pode pensar que é possível em outro estado. O grupo deve tomar uma decisão consciente, aplicá-la de forma consistente e justificá-la no relatório.

2. GERAÇÃO DE PDF

A geração do laudo em PDF é o único requisito técnico desta etapa que vai além do CRUD convencional. O grupo está implementando um sistema que produz um documento médico formal — e esse documento deve ter aparência e conteúdo à altura dessa responsabilidade.

CONTEÚDO OBRIGATÓRIO DO PDF

O laudo em PDF deve conter, no mínimo, as seguintes informações claramente organizadas:

- Nome completo do paciente.
- Nome completo do médico e seu CRM.
- Tipo de exame realizado (`examType`).
- Resultado do exame (`result`).
- Data de realização do exame (`endedAt` do atendimento).
- Data de emissão do laudo (`issuedAt`).
- Código de validação (`validationCode`) — em destaque, com instrução de como usá-lo para verificar a autenticidade.

CABEÇALHO HTTP DA RESPOSTA

O *endpoint* `GET /reports/{id}/pdf` deve retornar a resposta com os seguintes cabeçalhos:

- `Content-Type: application/pdf`
- `Content-Disposition: attachment; filename="laudo-{validationCode}.pdf"`

O `Content-Disposition` com `attachment` instrui o navegador a oferecer o *download* do arquivo em vez de tentar exibi-lo inline. O `filename` usa o `validationCode` para identificar unicamente o arquivo — evitando que múltiplos laudos baixados pelo mesmo cliente tenham o mesmo nome de arquivo.

SEPARAÇÃO DE RESPONSABILIDADES

A lógica de geração do PDF deve estar separada da lógica de negócio do `ReportsModule`. Um `PdfService` dedicado — que recebe os dados necessários e retorna o *buffer* do PDF — isola a dependência da biblioteca de geração e facilita a troca de biblioteca no futuro sem impactar o restante do sistema.



REFLEXÃO

Como o `PdfService` deve receber os dados do laudo?

O `PdfService` precisa de informações que estão em múltiplas entidades — o laudo em si, o exame associado, o paciente, o médico. Como esses dados chegam ao `PdfService`? Uma abordagem é passar o objeto `Report` completo — com suas relações carregadas via eager loading ou joins explícitos. Outra é criar um DTO específico — `LaudoPdfDto` — que consolida exatamente os campos necessários para o PDF, sem expor a estrutura interna das entidades ao `PdfService`. A segunda abordagem é mais coesa e mais fácil de testar, mas exige um passo adicional de mapeamento no `ReportsService`. O grupo deve escolher uma abordagem e justificar no relatório, considerando como a decisão impacta o acoplamento entre os módulos.



REFLEXÃO

O que fazer quando a biblioteca de PDF falha?

A geração de PDF pode falhar por diversos motivos, como falta de memória, inconsistência nos dados ou erros na biblioteca utilizada. Isso levanta a questão de como o sistema deve se comportar nesses cenários. Algumas possibilidades devem ser consideradas:

- O *endpoint* deve simplesmente retornar `500 Internal Server Error`?
- Ou é possível capturar a falha de forma mais específica e retornar um erro mais descritivo e útil?

O *Exception Filter* global implementado na Etapa 2 deve capturar exceções não tratadas e convertê-las para respostas no formato RFC 7807. No entanto, o grupo precisa verificar se os erros originados pela biblioteca de geração de PDF estão sendo corretamente interceptados e tratados nesse fluxo. Em especial, avalie se:

- As exceções da biblioteca estão sendo mapeadas adequadamente;
- As respostas retornadas seguem o padrão RFC 7807 definido para a API;
- O cliente recebe informações suficientes para entender a falha sem exposição de detalhes internos sensíveis.

Por fim, registre no relatório como os erros de geração de PDF são tratados, incluindo quaisquer ajustes realizados no fluxo de tratamento de exceções.

3. ATOMICIDADE DAS OPERAÇÕES CRÍTICAS

Esta etapa introduz operações que envolvem múltiplas escritas no banco de dados que devem ser tratadas como uma unidade indivisível — ou todas ocorrem, ou nenhuma ocorre.

CRIAÇÃO DE ATENDIMENTO

Ao criar um atendimento, duas operações ocorrem: o registro de `Appointment` é criado e o `Schedule` associado tem seu status atualizado para `COMPLETED`. Essas duas operações devem ser envolvidas em uma transação explícita. Se o registro do `Appointment` for criado mas a atualização do `Schedule` falhar, o sistema ficará em estado inconsistente — um agendamento confirmado sem atendimento correspondente.

EMISSÃO DE LAUDO

Ao emitir um laudo, o `validationCode` é gerado e o registro de `Report` é criado. Se a geração do PDF for feita e armazenada nessa mesma operação, ela também deve fazer parte da transação. O grupo deve investigar como o TypeORM gerencia transações no NestJS e implementar as transações explícitas necessárias.



REFLEXÃO

Todas as operações críticas desta etapa foram identificadas?

As duas operações acima são as mais óbvias em termos de atomicidade. Mas há outras operações nesta etapa que envolvem múltiplas escritas e que podem deixar o sistema em estado inconsistente se uma das etapas falhar? O grupo deve percorrer todos os fluxos de criação e atualização desta etapa e identificar sistematicamente quais envolvem múltiplas escritas. Para cada uma, deve decidir se uma transação explícita é necessária e como implementá-la no TypeORM. A lista de operações identificadas e as decisões correspondentes devem constar no relatório.

4. SEGURANÇA DOS DADOS CLÍNICOS

As entidades desta etapa contêm dados de saúde de pacientes — informações altamente sensíveis. A segurança implementada na Etapa 2 — autenticação JWT, autorização por perfil, controle por recurso — cobre a maioria dos cenários. Esta seção complementa com considerações específicas para os dados clínicos.

CAMPOS SENSÍVEIS NAS RESPOSTAS

O `result` de um `Exam` é um dado clínico sensível. O `diagnosis` e a `prescription` de um `MedicalRecord` são ainda mais sensíveis. O grupo deve revisar os DTOs de resposta de todos os *endpoints* que expõem essas informações e verificar se elas são retornadas apenas para os perfis autorizados — não apenas no *guard*, mas no próprio DTO de resposta.

O ENDPOINT PÚBLICO DE VALIDAÇÃO:

O `GET /reports/validate/{code}` é acessível sem autenticação. Isso significa que qualquer pessoa com o `validationCode` pode chamar esse *endpoint*. O grupo deve garantir que a resposta desse *endpoint* contenha

apenas o mínimo necessário para confirmar a autenticidade do laudo — sem expor dados clínicos do paciente que não sejam necessários para a validação.



REFLEXÃO

Há dados que o *endpoint* público de validação nunca deve retornar?

O `result` do exame é o dado mais sensível associado a um laudo. Faz sentido retorná-lo no *endpoint* público de validação? Uma seguradora que valida um laudo precisa saber o resultado detalhado do exame, ou apenas que o laudo é autêntico e pertence a determinado paciente? O grupo deve definir o contrato de resposta do *endpoint* público considerando o princípio do mínimo necessário — retornar apenas o que é indispensável para o propósito da validação — e justificar cada campo incluído e excluído no relatório.

5. QUALIDADE DO CÓDIGO DESTA ETAPA

Os requisitos de qualidade estabelecidos nas etapas anteriores — identificadores em inglês, métodos com responsabilidade única, comentários que explicam intenção — continuam válidos. Esta etapa adiciona uma consideração específica em relação à complexidade das regras de negócio.

REGRAS DE NEGÓCIO ENCADEADAS

Esta etapa tem a maior concentração de regras encadeadas do projeto. Um laudo depende de um exame encerrado com resultado. Um prontuário depende de um atendimento encerrado. Um procedimento especializado depende de autorização antes de ser realizado. Essas cadeias de dependência, implementadas como verificações sequenciais nos *services*, tendem a crescer e se tornar difíceis de ler se não forem organizadas com cuidado.

O grupo deve estruturar essas verificações de forma que a intenção seja clara — cada verificação deve ser expressiva o suficiente para comunicar por que está sendo feita, não apenas o que está verificando. Métodos auxiliares privados com nomes descritivos — `ensureAppointmentIsFinished()`, `ensureExamHasResult()`, `ensureNoActiveReport()` — são preferíveis a blocos de condicionais aninhados sem contexto.



REFLEXÃO

Como organizar as verificações de pré-condição nos *services* desta etapa?

Cada operação desta etapa possui um conjunto de pré-condições que devem ser validadas antes da execução. A emissão de um laudo, por exemplo, envolve pelo menos quatro verificações: o atendimento existe, é do tipo `Exam`, está no estado `FINISHED` e possui o campo `result` preenchido. Diante disso, surge uma questão de organização: *como estruturar essas validações dentro do service?*

Algumas abordagens possíveis incluem:

- Um único bloco sequencial de validações dentro do método principal;
- Extração dessas regras para métodos privados auxiliares, separando cada responsabilidade;
- Criação de um objeto de política (ou *policy*) que encapsula as regras de elegibilidade de forma reutilizável.

Cada abordagem tem impactos diretos na legibilidade, reutilização e facilidade de manutenção do código, especialmente quando novas regras forem adicionadas ao longo do tempo.

A organização escolhida deve ser aplicada de forma consistente em todos os *services* desta etapa e devidamente justificada no relatório, considerando sua escalabilidade e clareza estrutural.

6. PREPARAÇÃO PARA PRODUÇÃO — REFLEXÃO FINAL

Esta é a última entrega do projeto. O sistema está funcionalmente completo — mas funcional não significa pronto para produção. O grupo deve realizar uma análise honesta do que seria necessário para que o SGCM pudesse ser usado em um ambiente real, com usuários reais e dados reais de pacientes.

Essa análise não requer implementação adicional — ela pertence ao relatório final como uma seção de reflexão sobre o trabalho desenvolvido ao longo do semestre.

ASPECTOS A CONSIDERAR NA ANÁLISE

Banco de dados: O SQLite foi adequado para o desenvolvimento, mas tem limitações significativas para produção — sem suporte a múltiplas conexões concorrentes, sem replicação, sem backup automático. O que seria necessário para migrar para PostgreSQL ou MySQL? As decisões de herança tomadas ao longo do projeto facilitam ou dificultam essa migração?

Segurança: O sistema implementa autenticação JWT e controle de acesso por perfil. Mas há vetores de ataque que não foram cobertos — rate limiting no *login*, proteção contra injeção via parâmetros de *query*, headers de segurança HTTP. O grupo deve identificar os principais riscos de segurança do sistema atual e descrever o que seria implementado em produção para mitigá-los.

Performance: As *queries* de listagem com múltiplos filtros e as *queries* de agregação dos relatórios funcionam bem com o volume de dados de exemplo. Como elas se comportariam com milhares de atendimentos? Há índices no banco que deveriam ser criados? Há *queries* que deveriam ser otimizadas?

Observabilidade: O sistema tem *logging* básico implementado na Etapa 2. Em produção, seria necessário centralizar os logs, definir níveis de severidade, configurar alertas e implementar rastreamento distribuído. O que a infraestrutura atual de *logging* precisaria evoluir para suportar isso?



REFLEXÃO

O que o grupo aprendeu ao longo do semestre que mudaria se começasse o projeto novamente?

Esta é a reflexão mais importante do relatório final — e aquela que mais diferencia grupos que apenas entregaram código de grupos que realmente evoluíram ao longo do projeto. Neste ponto, o grupo deve analisar o projeto como um todo e responder de forma crítica às seguintes questões:

- Quais decisões tomadas na Etapa 1 geraram dificuldades ou retrabalho nas etapas seguintes?
- Há escolhas de modelagem que, com o conhecimento atual, seriam feitas de outra forma?
- Existem padrões de organização de código que poderiam ter sido adotados desde o início para evitar complexidade futura?
- Quais conceitos pareciam claros na teoria, mas só foram plenamente compreendidos na prática?

Essa análise deve ir além da descrição do que foi feito: ela deve evidenciar maturidade técnica e compreensão dos impactos arquiteturais das decisões tomadas ao longo do desenvolvimento.

Uma reflexão honesta sobre acertos, erros e aprendizados é o principal valor deste trabalho — e o resultado mais significativo do que foi construído ao longo do semestre.

DOCUMENTAÇÃO FINAL DA API

Esta é a última entrega do projeto — e a documentação Swagger deve refletir isso. Não é aceitável que o sistema esteja completo funcionalmente mas a documentação cubra apenas os *endpoints* da última etapa. O Swagger desta entrega final deve ser a fonte de verdade completa da API — cobrindo todas as três etapas, com o nível de detalhe e a qualidade esperados de um sistema em produção.

Um desenvolvedor que abre o Swagger pela primeira vez — sem ter lido nenhum dos documentos de especificação — deve conseguir entender o que o sistema faz, como se autenticar, quais *endpoints* existem, o que cada um espera receber e o que retorna em cada situação. Esse é o critério de qualidade da documentação final.

REVISÃO DA DOCUMENTAÇÃO EXISTENTE

Antes de documentar os novos *endpoints* desta etapa, o grupo deve revisar a documentação gerada nas etapas anteriores. Mudanças introduzidas ao longo do projeto podem ter tornado documentação anterior desatualizada — um *endpoint* que mudou de comportamento, um DTO que ganhou novos campos, um erro que passou a ter uma mensagem diferente.

O grupo deve percorrer cada *endpoint* das Etapas 1 e 2 no Swagger e verificar:

- O exemplo de requisição ainda corresponde ao DTO atual?
- O exemplo de resposta de sucesso inclui o envelope do *Transform Interceptor* introduzido na Etapa 2?
- Os exemplos de resposta de erro seguem o formato RFC 7807?
- Os campos adicionados ao longo do projeto — `createdBy`, `cancelledBy`, `isActive` — aparecem nos exemplos de resposta?
- A descrição da operação reflete o comportamento atual, incluindo as restrições de acesso implementadas na Etapa 2?

Documentação desatualizada é documentação incorreta — e documentação incorreta é pior do que documentação ausente, porque induz o consumidor da API a erro.



REFLEXÃO

Como o grupo garantiu que a documentação se manteve atualizada ao longo das três etapas?

A documentação tende a ficar desatualizada quando é tratada como uma tarefa separada do desenvolvimento — algo que se faz no final, antes da entrega. O grupo deve refletir: a política estabelecida na Etapa 2 — atualizar o Swagger como critério de aprovação nos *pull requests* — foi seguida consistentemente? Houve momentos em que a documentação ficou para depois e acumulou dívida? O que o grupo faria diferente para manter a documentação sincronizada com o código ao longo de todo o projeto? Registre essa reflexão no relatório final.

DOCUMENTANDO OS NOVOS *ENDPOINTS*

Os *endpoints* desta etapa introduzem casos de documentação que não foram enfrentados anteriormente. O grupo deve resolvê-los com cuidado.

HIERARQUIAS DE HERANÇA NOS DTOS

Os *endpoints* `POST /appointments` e `POST /appointments/{id}/procedures` aceitam corpos diferentes dependendo do `type` informado. O Swagger deve comunicar essa variação de forma clara — um consumidor que não sabe que `POST /appointments` com `type = FOLLOW_UP` exige `originAppointmentId` pode ficar horas depurando um erro `400`.

O pacote `@nestjs/swagger` oferece o decorator `@ApiExtraModels()` combinado com `oneOf` para documentar DTOs discriminados — permitindo que o Swagger exiba os diferentes schemas possíveis e indique que apenas um deve ser enviado por vez. O grupo deve investigar como implementar essa documentação e aplicá-la aos *endpoints* com hierarquias.

```
@ApiBody({
  schema: {
    oneOf: [
      { $ref: getSchemaPath(CreateConsultationDto) },
      { $ref: getSchemaPath(CreateExamDto) },
      { $ref: getSchemaPath(CreateFollowUpDto) },
    ],
  },
})
```

O *ENDPOINT* DE *DOWNLOAD* DE *PDF*

O `GET /reports/{id}/pdf` retorna um arquivo binário — não JSON. O Swagger precisa comunicar isso claramente para que o consumidor não tente interpretar a resposta como JSON. O `@ApiResponse()` deve declarar o tipo de conteúdo correto:

```
@ApiResponse({
  status: 200,
  description: 'Laudo em formato PDF.',
  content: {
    'application/pdf': {
      schema: {
        type: 'string',
        format: 'binary',
      },
    },
  },
})
```

```
})
```

O grupo deve também documentar o cabeçalho `Content-Disposition` que instrui o navegador a realizar o *download* — usando `@ApiResponse()` ou descrevendo-o na seção de descrição do *endpoint*.

O ENDPOINT PÚBLICO DE VALIDAÇÃO

O `GET /reports/validate/{code}` é o único *endpoint* sem autenticação além dos de *login* e *refresh*. A documentação deve comunicar isso de forma inequívoca — sem `@ApiBearerAuth()`, com uma descrição que explique explicitamente que este *endpoint* não requer autenticação e pode ser chamado por qualquer pessoa que possua o código de validação.

A descrição deve também explicar o propósito clínico do *endpoint* — permitir que terceiros verifiquem a autenticidade de laudos emitidos pelo sistema — para que o consumidor entenda por que este *endpoint* existe e como ele deve ser usado.

OS RELATÓRIOS ADMINISTRATIVOS

Os relatórios retornam dados agregados — não listas paginadas. A documentação deve deixar claro que esses *endpoints* não suportam os parâmetros `page`, `limit` e `sort`, e que a estrutura de resposta é diferente dos *endpoints* de listagem. Os exemplos de resposta devem mostrar dados agregados realistas — não apenas `{}` ou estruturas genéricas.

```
@ApiResponse({
  status: 200,
  description: 'Relatório de agendamentos por status e modalidade.',
  schema: {
    example: {
      data: {
        byStatus: {
          PENDING: 12,
          CONFIRMED: 34,
          CANCELLED: 8,
          COMPLETED: 67,
        },
        byType: {
          IN_PERSON: 45,
          ONLINE: 52,
          HOME: 24,
        },
        total: 121,
        period: {
          startDate: '2025-09-01',
          endDate: '2025-09-30',
        },
      },
    },
  },
})
```

```

    },
    meta: {
      timestamp: '2025-09-30T18:00:00Z',
      path: '/admin/reports/schedules',
    },
  },
},
})

```



REFLEXÃO

Os exemplos de resposta dos relatórios devem usar dados fictícios ou dados do banco de exemplo?

Os exemplos de resposta no Swagger devem ser realistas — não `0` para todos os contadores, não strings genéricas. Uma abordagem é usar dados fictícios que parecem plausíveis para uma clínica em funcionamento. Outra é usar dados que correspondem exatamente ao que o banco de exemplo contém — o que permite ao avaliador verificar o comportamento real do *endpoint* e comparar com a documentação. O grupo deve decidir qual abordagem adotar e garantir que os exemplos sejam internamente consistentes — os totais devem bater, os IDs referenciados devem existir no banco. Registre a decisão no relatório.

ATUALIZAÇÃO DA VERSÃO E DESCRIÇÃO

Esta é a entrega final — a versão do Swagger deve refletir a completude do sistema. O `setVersion()` no `DocumentBuilder` deve ser atualizado para `3.0` ou para o esquema de versionamento que o grupo definiu ao longo do projeto.

A descrição geral do Swagger deve ser reescrita para descrever o sistema completo — não apenas a última etapa. Um desenvolvedor que lê a descrição deve entender o que o SGCM é, quais são seus módulos principais, como a autenticação funciona e onde encontrar a documentação de cada funcionalidade.

```

const config = new DocumentBuilder()
  .setTitle('SGCM – Sistema de Gestão de Clínica Médica')
  .setDescription(`
    API completa para gerenciamento de clínica médica, cobrindo o ciclo clínico do agendame

    ## Autenticação
    A maioria dos endpoints requer autenticação JWT. Para autenticar:
    1. Faça login em `POST /auth/login` com e-mail e senha.
    2. Copie o `accessToken` retornado.
    3. Clique em Authorize no topo desta página e insira o token no formato `Bearer {t
  `)

```

```
## Perfil de Usuário
- **Admin**: acesso irrestrito a todos os recursos.
- **Doctor**: acesso aos próprios atendimentos, procedimentos e laudos.
- **Patient**: acesso aos próprios agendamentos, prontuários e laudos.

## Endpoint Público
O endpoint `GET /reports/validate/{code}` não requer autenticação e pode ser usado
por qualquer pessoa para verificar a autenticidade de um laudo emitido pelo sistema.

## Credenciais de Teste
Consulte o README.md do repositório para as credenciais dos usuários de exemplo.
`)
.setVersion('3.0')
.addBearerAuth(
  {
    type: 'http',
    scheme: 'bearer',
    bearerFormat: 'JWT',
    description: 'token JWT obtido em POST /auth/login',
  },
  'access-token',
)
.build();
```



REFLEXÃO

A descrição acima está completa para o sistema que o grupo construiu?

A descrição acima é um ponto de partida. O grupo deve expandi-la para refletir as decisões específicas que tomou ao longo do projeto — convenções de nomenclatura adotadas, comportamentos não óbvios, limitações conhecidas do sistema. Um consumidor que lê a descrição e ainda tem dúvidas sobre como usar a API encontrará as respostas na documentação dos *endpoints* individuais — mas decisões transversais ao sistema todo pertencem à descrição geral. O grupo deve revisar a descrição final e verificar se ela está completa para o sistema que foi construído — não para o sistema genérico descrito neste documento.

TAGS E AGRUPAMENTO

O Swagger agrupa *endpoints* por *tags* — cada `@ApiTags()` define a qual grupo um conjunto de *endpoints* pertence. Com o sistema completo, o número de *endpoints* é significativo, e o agrupamento deve ser intuitivo para

quem navega pela documentação.

O grupo deve revisar as *tags* definidas ao longo das etapas anteriores e garantir que o agrupamento final seja coerente. As *tags* sugeridas para o sistema completo são:

TAG	ENDPOINTS AGRUPADOS
Auth	/auth/login, /auth/refresh, /auth/me, /auth/logout
Users	/users/*
Doctors	/doctors/*
Patients	/patients/*
Specialties	/specialties/*
Schedules	/schedules/*
Appointments	/appointments/*
Procedures	/procedures/*
Medical Records	/records/*
Reports	/reports/*
Admin	/admin/*



REFLEXÃO

O agrupamento por *tags* acima reflete a organização que o grupo adotou?

As *tags* sugeridas acima seguem a organização por recurso — cada entidade tem sua própria *tag*. Mas o grupo pode ter adotado uma organização diferente ao longo do projeto — por exemplo, agrupando **Doctors** e **Patients** dentro de **Users**. O grupo deve verificar se o agrupamento atual no Swagger é intuitivo para um consumidor externo e fazer os ajustes necessários. Considere também: os *endpoints* de relatório de médicos — `GET /doctors/{id}/appointments`, `GET /doctors/{id}/records` — pertencem à *tag* **Doctors** ou às *tags* **Appointments** e **Medical Records**? Não há uma resposta única — mas a escolha deve ser consistente e justificada no relatório.

QUALIDADE DA DOCUMENTAÇÃO FINAL

O grupo deve realizar uma revisão final completa do Swagger antes de submeter a entrega. Esta revisão deve ser feita como se o grupo fosse um desenvolvedor externo tentando integrar com a API pela primeira vez — sem

conhecimento prévio do sistema.

Autenticação:

- ☐ O botão "Authorize" está funcional — ao inserir o *token*, todos os *endpoints* protegidos o enviam automaticamente.
- ☐ Todos os *endpoints* protegidos têm `@ApiBearerAuth('access-token')`.
- ☐ O *endpoint* público de validação não tem `@ApiBearerAuth()`.
- ☐ A descrição geral explica o fluxo de autenticação de forma clara.

Cobertura:

- ☐ Todos os *endpoints* das três etapas estão documentados.
- ☐ Nenhum *endpoint* foi esquecido — compare com as tabelas de *endpoints* obrigatórios de cada etapa.
- ☐ *Endpoints* adicionais implementados pelo grupo também estão documentados.

Qualidade dos exemplos:

- ☐ Nenhum exemplo usa valores genéricos — `"string"`, `0`, `true` sem contexto.
- ☐ Os exemplos de requisição são realistas para o domínio clínico.
- ☐ Os exemplos de resposta de sucesso incluem o envelope `{ data, meta }` do *Transform Interceptor*.
- ☐ Os exemplos de resposta de erro seguem o formato RFC 7807.
- ☐ Os exemplos de *token* JWT nos campos de resposta estão no formato correto — não são strings genéricas.

Hierarquias:

- ☐ Os *endpoints* com hierarquias de herança — `POST /appointments`, `POST /appointments/{id}/procedures` — documentam os diferentes schemas possíveis com `oneOf`.

- ☐ Cada subtipo tem um exemplo distinto que ilustra seus atributos específicos.

Casos especiais:

- ☐ O *endpoint* de PDF documenta `Content-Type: application/pdf` e o cabeçalho `Content-Disposition`.
- ☐ O *endpoint* público de validação documenta que não requer autenticação e explica seu propósito.
- ☐ Os relatórios administrativos documentam que não suportam paginação e mostram a estrutura de dados agregados.
- ☐ Os erros `401`, `403`, `404` e `409` estão documentados nos *endpoints* que os retornam.

Consistência:

- ☐ As *tags* agrupam os *endpoints* de forma intuitiva e consistente.
- ☐ A descrição de cada operação é clara o suficiente para um desenvolvedor externo entender o que o *endpoint* faz.
- ☐ Campos sensíveis — `password`, `refreshToken` — não aparecem em nenhum exemplo de resposta.
- ☐ A versão do Swagger (`setVersion()`) reflete a entrega final.



REFLEXÃO

A documentação final representa fielmente o sistema construído ao longo do semestre?

Esta é a pergunta mais importante sobre a documentação final. O grupo deve abrir o Swagger e testar o uso da API exclusivamente a partir da documentação — sem consultar o código-fonte. Durante esse processo, verifique se é possível realizar, apenas com as informações documentadas:

- O *login*;
- A criação de um agendamento;
- A criação de um atendimento a partir de um agendamento;
- A emissão de um laudo;
- A validação do laudo no *endpoint* público;
- E quaisquer outras funcionalidades.

Se em qualquer uma dessas etapas a documentação não for suficiente para orientar corretamente a operação — por falta de exemplos, campos mal descritos ou ausência de informações importantes — isso indica uma lacuna que precisa ser corrigida. O grupo deve, então:

- Identificar os pontos de falha ou ambiguidade na documentação;
- Corrigir o Swagger para torná-lo autossuficiente;
- Garantir que qualquer pessoa consiga utilizar a API apenas a partir dele.

Por fim, registre no relatório o resultado dessa revisão, incluindo o que foi encontrado e quais melhorias foram aplicadas.

ENTREGA

BRANCH E FLUXO DE ENTREGA

O desenvolvimento desta etapa deve ocorrer na *branch* `stage-3`, criada a partir de `main` após o merge de `stage-2`. O trabalho do dia a dia deve ser feito em *branches* `feature/*` criadas a partir de `stage-3`.

GIT BRANCHES

```
main HEAD origin/main  
  
stage-1 (merged)  
stage-2 (merged)  
stage-3  
  feature/appointments-module  
  feature/procedures-module  
  feature/medical-records-module  
  feature/reports-module  
  feature/pdf-generation  
  feature/public-validation-endpoint  
  feature/admin-reports  
  feature/swagger-final  
  feature/...
```

Ao concluir a etapa, `stage-3` deve ser mergeada em `main` por meio de um *pull request* final — que representa a entrega oficial do projeto completo. As *branches* `main` e `stage-3` são protegidas e não aceitam push direto.



REFLEXÃO

Como o grupo planeja a ordem de implementação desta etapa?

As dependências entre os módulos desta etapa determinam a ordem em que devem ser implementados. O `AppointmentsModule` deve existir antes do `ProceduresModule`, do `MedicalRecordsModule` e do `ReportsModule`. O `ReportsModule` depende de `AppointmentsModule` para validar o exame antes de emitir o laudo. O `AdminModule` depende de praticamente todos os outros módulos. O grupo deve construir um grafo de dependências antes de criar as *branches* `feature/*` e definir a sequência de implementação que minimize bloqueios — situações em que um membro não pode progredir porque depende de código que outro ainda não terminou. Registre esse plano de implementação no relatório e compare, ao final, com o que realmente aconteceu.

O QUE DEVE ESTAR NO REPOSITÓRIO NA DATA DE ENTREGA

A entrega será avaliada a partir do estado da *branch* `main` após o merge de `stage-3`. O repositório deve conter, de forma completa e funcional, tudo que foi construído ao longo das três etapas.

Código — novos módulos desta etapa:

- `AppointmentsModule` com hierarquia de herança implementada, ciclo de vida controlado e integração com `SchedulesModule`.
- `ProceduresModule` com hierarquia de herança implementada e ciclo de autorização de procedimentos especializados funcionando.
- `MedicalRecordsModule` com criação controlada por estado do atendimento, proteção contra exclusão e rastreabilidade de atualizações.
- `ReportsModule` com emissão de laudos, geração de PDF, revogação com rastreabilidade e *endpoint* público de validação.
- `AdminModule` com os quatro relatórios administrativos — agendamentos, atendimentos, procedimentos e ocupação de agenda.
- Todos os novos *endpoints* protegidos com os *guards* da Etapa 2 — exceto `GET /reports/validate/{code}`.
- Controle de acesso por recurso implementado nos *services* para todos os *endpoints* identificados na tabela de controle de acesso.
- Campos de rastreabilidade preenchidos automaticamente com o usuário autenticado onde aplicável — `authorizedBy` em `SpecializedProcedure`, `lastUpdatedBy` em `MedicalRecord`, `revokedBy` em `Report`.
- Operações críticas — criação de atendimento, emissão de laudo — envolvidas em transações explícitas.

Código — revisão e consolidação:

- Toda a infraestrutura das Etapas 1 e 2 funcionando corretamente para os novos *endpoints* — *middleware* de *logging*, *Transform Interceptor*, *Exception Filter*.
- O *endpoint* de PDF tratado corretamente pelo *Transform Interceptor* — sem tentar envolver o *buffer* binário no envelope JSON.

- Dependências circulares entre módulos ausentes — verificado e documentado no relatório.

Banco de dados:

- Arquivo `database.db` atualizado com todas as entidades das três etapas.
- Schema compatível com as entidades implementadas — sem tabelas fantasmas ou colunas órfãs.
- Dados de exemplo cobrindo os fluxos obrigatórios das três etapas:
 - Ao menos um atendimento de cada tipo — `Consultation`, `Exam`, `FollowUp` — em estados diferentes.
 - Ao menos um `SimpleProcedure` e um `SpecializedProcedure` em cada estado de autorização.
 - Ao menos um `MedicalRecord` criado para um atendimento encerrado.
 - Ao menos um `Report` com status `ACTIVE` e outro com status `REVOKED` — com `validationCode` reais documentados no `README.md`.
 - Dados suficientes para que os relatórios administrativos retornem métricas não nulas.

Documentação:

- Swagger final cobrindo todos os *endpoints* das três etapas.
- Versão atualizada do Swagger refletindo a entrega final.
- Descrição geral do Swagger descreve o sistema completo com guia de uso.
- *Endpoint* público de validação sem `@ApiBearerAuth()`.
- *Endpoint* de PDF com `Content-Type: application/pdf` documentado.
- Hierarquias de herança documentadas com `oneOf` e exemplos distintos por subtipo.
- Relatórios administrativos com exemplos de resposta realistas.
- Toda a documentação das Etapas 1 e 2 revisada e atualizada.

Arquivos obrigatórios:

- `README.md` final com instruções completas de instalação, todas as variáveis de ambiente necessárias, credenciais de teste e `validationCode` de laudos de exemplo para testar o *endpoint* público.
- `report.md` final com todas as seções obrigatórias das três etapas — atualizado, não reescrito.
- `diagram.png` ou `diagram.puml` final e completo refletindo todas as entidades implementadas.
- `.env.example` com todas as variáveis de ambiente necessárias para executar o sistema.
- `.gitignore` garantindo que `.env` e o arquivo `.db` gerado localmente não sejam conflitados com o `.db` de exemplo.

README.md FINAL

O `README.md` desta entrega é o documento de apresentação do projeto completo. Ele deve ser suficientemente claro para que qualquer pessoa — inclusive o professor durante a avaliação — consiga instalar, configurar e executar o sistema sem fazer nenhuma pergunta.

O `README.md` final deve conter, no mínimo:

- **Descrição do projeto:** o que é o SGCM, o que ele faz e quais são seus módulos principais. Duas ou três frases são suficientes — o Swagger complementa com os detalhes.

- **Tecnologias e versões:** NestJS, TypeScript, TypeORM, SQLite, Node.js — com as versões efetivamente utilizadas.
- **Instalação e execução:**

```
# clonar o repositório
git clone https://github.com/grupo/sbcm.git
cd sgcm
# instalar dependências
npm install
# configurar variáveis de ambiente
cp .env.example .env
# editar .env com os valores necessários
# iniciar o servidor
npm run start
```

- **Variáveis de ambiente:** todas as variáveis necessárias com descrição do que cada uma representa.
- **Credenciais de teste:** tabela com e-mail, senha e perfil de cada usuário de exemplo presente no banco.
- **Exemplos para testar o endpoint público:** ao menos dois `validationCode` reais — um de laudo `ACTIVE` e outro de laudo `REVOKED` — para que o avaliador possa testar `GET /reports/validate/{code}` sem precisar criar dados manualmente.
- **Link do Swagger:** `http://localhost:{PORT}/api`



REFLEXÃO

O `README.md` atual está completo para a entrega final?

O `README.md` foi atualizado a cada etapa ao longo do semestre. Antes de submeter a entrega final, o grupo deve lê-lo do início ao fim como se fosse um desenvolvedor externo clonando o repositório pela primeira vez. Há instruções que assumem conhecimento prévio do projeto? Há variáveis de ambiente que foram adicionadas ao longo do projeto mas não estão documentadas? As credenciais de teste estão atualizadas — com senhas que realmente funcionam no banco entregue? O link do Swagger aponta para a porta correta? Cada lacuna encontrada deve ser corrigida antes da submissão.

RELATÓRIO FINAL

O relatório é o documento que acompanha o sistema ao longo de todo o semestre. Esta entrega marca seu ponto final — não como uma seção adicionada ao final, mas como a conclusão de um documento que evoluiu junto com o projeto.

O relatório final deve ser coeso: as decisões da Etapa 1 devem ser referenciadas quando influenciam decisões da

Etapa 3. As lições aprendidas ao longo do projeto devem aparecer nas reflexões finais. Um relatório que parece três documentos separados colados em sequência demonstra menos do que um relatório que conta a história de um projeto que evoluiu conscientemente.

Seções obrigatórias desta entrega:

1. **Integrantes e contribuições** — Descrição da contribuição de cada membro nesta etapa, incluindo a estratégia de divisão de trabalho e como as dependências entre módulos foram gerenciadas. Esta seção deve ser lida em conjunto com as seções equivalentes das etapas anteriores — ela completa a narrativa de como o grupo trabalhou ao longo do semestre.
2. **Diagrama de classes final** — Versão completa e atualizada cobrindo todas as entidades implementadas nas três etapas. O diagrama deve refletir o sistema conforme foi construído — não o que foi planejado. Se o grupo se afastou do diagrama base em algum ponto, as diferenças devem ser identificadas e justificadas.
3. **Decisões técnicas desta etapa** — Cobrindo todas as reflexões marcadas com  neste documento. Cada decisão deve ser descrita com a alternativa considerada, a escolha feita e sua justificativa. Decisões que se relacionam com escolhas feitas nas etapas anteriores devem referenciá-las explicitamente.
4. **Tabela de controle de acesso final** — Versão consolidada e completa cobrindo todos os *endpoints* das três etapas — com perfis autorizados e indicação de controle por recurso. Esta tabela é o mapa de segurança do sistema e deve ser consistente com o que foi implementado no código.
5. **Mapa de dependências entre módulos** — Diagrama ou tabela mostrando as dependências entre todos os módulos do sistema — incluindo os das etapas anteriores. Deve identificar como as dependências circulares foram evitadas e onde o `forwardRef()` foi necessário, se aplicável.
6. **Limitações conhecidas do sistema** — Análise honesta das limitações do SGCM — não apenas desta etapa, mas do sistema como um todo. Limitações de segurança, de performance, de modelagem de dados, de escalabilidade. O que seria necessário para superar cada limitação em um ambiente de produção real.
7. **Reflexão sobre herança no banco de dados** — Síntese das quatro decisões de herança tomadas ao longo do projeto — `User`, `Schedule`, `Appointment`, `Procedure`. O que o grupo aprendeu sobre herança no banco de dados? Qual estratégia se mostrou mais adequada para cada tipo de hierarquia? O que faria diferente em um próximo projeto?
8. **Dificuldades e aprendizados desta etapa** — Principais dificuldades técnicas e organizacionais desta etapa e como foram resolvidas.
9. **Reflexão final** — O que o grupo construiu ao longo do semestre, o que aprendeu de mais significativo e o que faria diferente se começasse o projeto novamente. Esta seção é a mais livre do relatório — e costuma ser a mais reveladora sobre o nível de compreensão do grupo.

LISTA DE VERIFICAÇÃO FINAL

Esta lista cobre os pontos críticos de todas as três etapas. O grupo deve revisá-la integralmente antes de submeter — não apenas os itens desta etapa.

Funcionamento geral:

- ☐ O projeto instala (`npm install`) e inicia (`npm run start`) sem erros a partir das instruções do `README.md`.

☐ O Swagger está acessível em `/api` após iniciar o projeto.

☐ O banco de dados contém dados de exemplo suficientes para testar todos os fluxos obrigatórios das três etapas.

Etapa 1 — verificação final:

☐ CRUD de usuários, especialidades e agendamentos funcionando com as regras de negócio da Etapa 1.

☐ Paginação e filtros funcionando em todos os *endpoints* de listagem.

☐ Formato de resposta com envelope `{ data, meta }` em todos os *endpoints* — incluindo os metadados do *Transform Interceptor* da Etapa 2.

Etapa 2 — verificação final:

☐ *Login, refresh, logout* e `GET /auth/me` funcionando corretamente.

☐ Todos os *endpoints* protegidos retornam `401` sem *token* e `403` para perfil sem permissão.

☐ O *endpoint* público de validação — `GET /reports/validate/{code}` — é acessível sem *token*.

☐ *Middleware* de *logging* registrando todas as requisições com os campos obrigatórios.

☐ *Exception Filter* retornando erros no formato RFC 7807 em todos os cenários.

Etapa 3 — atendimentos:

☐ Atendimentos são criados apenas a partir de agendamentos `CONFIRMED`.

☐ O agendamento tem status atualizado para `COMPLETED` atomicamente ao criar o atendimento.

☐ O tipo do atendimento não pode ser alterado após a criação.

☐ `FollowUp` valida que o `originAppointmentId` pertence ao mesmo paciente.

☐ Atendimentos encerrados não podem voltar ao estado `IN_PROGRESS`.

☐ Atendimentos não podem ser excluídos.

Etapa 3 — procedimentos:

☐ Procedimentos só podem ser adicionados a atendimentos `IN_PROGRESS`.

☐ `SpecializedProcedure` com `requiresAuthorization = true` inicia com `authorizationStatus = PENDING`.

☐ `authorizedBy` é preenchido automaticamente com o `Admin` autenticado.

☐ A transição `DENIED → PENDING` é bloqueada.

Etapa 3 — prontuários:

☐ Prontuários são criados apenas para atendimentos `FINISHED`.

☐ Cada atendimento tem no máximo um prontuário — `409` para tentativa de criar segundo.

☐ Prontuários não podem ser excluídos.

☐ `lastUpdatedBy` é preenchido automaticamente nas atualizações.

☐ Controle por recurso implementado para `Doctor` e `Patient`.

Etapa 3 — laudos:

☐ Laudos são emitidos apenas para `Exam` com `status = FINISHED` e `result` preenchido.

☐ `validationCode` é gerado automaticamente — nunca recebido do cliente.

☐ Apenas um laudo `ACTIVE` por exame — `409` para tentativa de emitir segundo.

☐ PDF gerado com todas as informações obrigatórias e `Content-Type: application/pdf`.

☐ *Endpoint* público `GET /reports/validate/{code}` acessível sem autenticação.

☐ Revogação preenche `revokedAt`, `revokedBy` e `revokedReason` — é irreversível.

☐ Após revogação, novo laudo pode ser emitido para o mesmo exame.

Etapla 3 — relatórios:

☐ Todos os quatro relatórios retornam `403` para `Doctor` e `Patient`.

☐ As métricas obrigatórias estão presentes e corretas em cada relatório.

☐ Filtro por intervalo de datas funciona — `startDate` posterior a `endDate` retorna `400`.

☐ Fórmula de taxa de ocupação documentada no Swagger e no relatório.

Documentação:

☐ Todos os *endpoints* das três etapas documentados no Swagger.

☐ Hierarquias documentadas com `oneOf` e exemplos por subtipo.

☐ *Endpoint* de PDF com `Content-Type: application/pdf` documentado.

☐ *Endpoint* público sem `@ApiBearerAuth()`.

☐ Relatórios com exemplos de dados agregados realistas.

☐ Nenhum exemplo usa valores genéricos como `"string"` ou `0`.

Repositório e relatório:

☐ A *branch* `stage-3` foi mergeada em `main` via *pull request* aprovado por todos os membros.

☐ O `README.md` final está completo — com credenciais de teste e `validationCode` de exemplo.

☐ O `report.md` contém todas as seções obrigatórias das três etapas de forma coesa.

- ☐ O diagrama de classes final está completo e consistente com o código.
- ☐ A tabela completa de controle de acesso cobre os *endpoints* das três etapas.
- ☐ O `.env` não está versionado — está no `.gitignore`.
- ☐ Todos os membros têm *commits* distribuídos ao longo do período desta etapa.

SUBMISSÃO

ITEM	DETALHE
Prazo	10 de junho
Canal de submissão	AVA — submeter apenas o link do repositório no GitHub
Branch avaliada	<code>main</code> — com <code>stage-3</code> mergeada via pull request
Visibilidade	Público ou privado, com <code>@lordantonelli</code> como colaborador

Esta é a entrega final do projeto. Não haverá resubmissão após o prazo — o estado da *branch* `main` no momento do prazo é o que será avaliado. Certifique-se de que o repositório está acessível, o banco contém dados de exemplo, o Swagger está completo, o `README.md` contém as credenciais de teste e os `validationCode` de exemplo, e o `report.md` está atualizado antes de submeter.

Submissões sem acesso ao repositório, sem banco de dados, sem dados de exemplo ou realizadas após o prazo não serão avaliadas.

CRITÉRIOS DE AVALIAÇÃO

A nota desta etapa varia de **0 a 10** e é atribuída ao grupo. Por ser a entrega final, a avaliação considera dois aspectos complementares: a correção e qualidade do que foi implementado nesta etapa, e a coerência do sistema como um todo — se o projeto evoluiu de forma consistente, se as decisões foram conscientes e se o grupo demonstra compreensão do que construiu ao longo do semestre.

Um sistema que funciona mas cujo relatório final não reflete as decisões tomadas, cujo Swagger não cobre todos os *endpoints* ou cujo banco não tem dados de exemplo que permitam testar os fluxos demonstra menos maturidade do que um sistema completo, documentado e pronto para ser usado por um avaliador externo sem necessidade de instrução adicional.

DISTRIBUIÇÃO DOS CRITÉRIOS

CRITÉRIO	PESO
1. Atendimentos e procedimentos	2,5
2. Prontuários e laudos	2,5
3. Relatórios administrativos	1,5
4. Documentação final da API	2,0
5. Relatório e decisões técnicas	1,5
Total	10,0

1. ATENDIMENTOS E PROCEDIMENTOS — 2,5 PONTOS

Este critério avalia se as duas hierarquias de herança mais complexas do sistema foram corretamente implementadas e se as regras de negócio que governam seu ciclo de vida foram respeitadas integralmente. É o critério de maior complexidade desta etapa — não apenas pela quantidade de regras, mas pela interdependência entre elas.

O avaliador verificará:

Hierarquias de herança:

- As estratégias de herança para **Appointment** e **Procedure** estão implementadas corretamente no banco de dados?
- A estratégia escolhida para cada hierarquia está justificada no relatório com base no perfil de consultas do domínio?
- Os subtipos carregam apenas os atributos que lhes pertencem — sem colunas desnecessárias ou nulas em

excesso?

Ciclo de vida dos atendimentos:

- Atendimentos são criados apenas a partir de `Schedule` com `status = CONFIRMED`?
- A atualização do `Schedule` para `COMPLETED` ocorre atomicamente com a criação do `Appointment` — em uma transação explícita?
- O `type` do atendimento é imutável após a criação?
- Um `FollowUp` valida corretamente que o `originAppointmentId` pertence ao mesmo `Patient` e não referencia a si mesmo?
- O encerramento via `PATCH /appointments/{id}/finish` é irreversível — o sistema bloqueia qualquer tentativa de reverter o status?
- Atendimentos não podem ser excluídos em nenhuma circunstância?

Ciclo de vida dos procedimentos:

- Procedimentos só podem ser adicionados a atendimentos `IN_PROGRESS`?
- `SpecializedProcedure` com `requiresAuthorization = true` inicia com `authorizationStatus = PENDING` — sem possibilidade de o cliente fornecer esse valor?
- O campo `authorizedBy` é preenchido automaticamente com o `Admin` autenticado — nunca recebido do cliente?
- A transição `DENIED → PENDING` é bloqueada de forma explícita?
- O controle por recurso está implementado corretamente para `Doctor` em todos os *endpoints* de procedimentos?

Penalizações proporcionais: herança implementada incorretamente ou sem justificativa no relatório, ausência de atomicidade na criação do atendimento, `FollowUp` sem validação do atendimento de origem, ciclo de autorização de procedimentos com transições incorretas, campos de rastreabilidade recebidos do cliente em vez de preenchidos automaticamente e controle por recurso ausente ou implementado no *controller* resultam em desconto proporcional dentro deste critério.

2. PRONTUÁRIOS E LAUDOS — 2,5 PONTOS

Este critério avalia se os documentos clínicos do sistema foram implementados com o rigor que seu caráter formal exige — prontuários imutáveis em termos de exclusão, laudos com código único de validação, PDF gerado corretamente e *endpoint* público funcionando sem autenticação.

O avaliador verificará:

Prontuários:

- Prontuários são criados apenas para `Appointment` com `status = FINISHED`?
- A restrição de um prontuário por atendimento está implementada — `409 Conflict` para tentativa de criar segundo?
- Prontuários não podem ser excluídos — o sistema retorna o status HTTP adequado com mensagem explicativa?
- O campo `lastUpdatedBy` é preenchido automaticamente com o usuário autenticado nas atualizações?

- O controle por recurso está implementado corretamente para `Doctor` e `Patient` em todos os *endpoints* de prontuário?
- A política de quais campos podem ser atualizados — e por quem — está implementada e documentada no relatório?

Emissão de laudos:

- Laudos são emitidos apenas para `Exam` com `status = FINISHED` e `result` preenchido — com mensagens de erro descritivas para cada condição não atendida?
- O `validationCode` é um UUID gerado automaticamente pelo sistema — nunca recebido do cliente, nunca nulo?
- A restrição de um único laudo `ACTIVE` por exame está implementada — `409 Conflict` para tentativa de emitir segundo?
- A emissão do laudo é tratada de forma atômica quando envolve múltiplas operações?

Geração de PDF:

- O PDF é gerado com todas as informações obrigatórias — nome do paciente, nome do médico, CRM, tipo de exame, resultado, data de emissão e `validationCode` em destaque?
- O *endpoint* retorna `Content-Type: application/pdf` e `Content-Disposition: attachment; filename="laudo-{validationCode}.pdf"`?
- O *Transform Interceptor* não tenta envolver o *buffer* de PDF no envelope JSON?
- O layout do PDF é legível e profissional — não um texto sem formatação?

Validação pública:

- O *endpoint* `GET /reports/validate/{code}` é acessível sem autenticação — o `JwtAuthGuard` global não o bloqueia?
- A resposta contém dados suficientes para confirmar a autenticidade sem expor informações clínicas sensíveis desnecessárias?
- Para laudos `REVOKED`, a resposta comunica claramente o status sem esconder a existência do registro?
- O comportamento para `validationCode` inexistente retorna `404` com mensagem adequada?

Revogação:

- A revogação preenche `revokedAt`, `revokedBy` e `revokedReason` automaticamente?
- A transição `ACTIVE → REVOKED` é irreversível — o sistema bloqueia tentativa de revogar laudo já revogado?
- O controle por recurso para `Doctor` — apenas laudos que emitiu — está implementado no *service*?
- Após revogação, um novo laudo pode ser emitido para o mesmo exame?

Penalizações proporcionais: laudos emitidos sem as condições obrigatórias, `validationCode` recebido do cliente ou gerado de forma não única, *endpoint* público bloqueado por autenticação, PDF sem as informações obrigatórias ou com `Content-Type` incorreto, revogação sem rastreabilidade completa, *Transform Interceptor* interferindo na resposta de PDF e controle por recurso ausente resultam em desconto proporcional dentro deste critério.

3. RELATÓRIOS ADMINISTRATIVOS — 1,5 PONTOS

Este critério avalia se os relatórios oferecem ao `Admin` métricas corretas, com filtros funcionais e controle de acesso adequado. A simplicidade aparente dos relatórios — são apenas operações de leitura — não deve obscurecer a importância da correção das métricas e da clareza da estrutura de resposta.

O avaliador verificará:

- Os quatro relatórios obrigatórios estão implementados e acessíveis?
- Todos os relatórios retornam `403` para `Doctor` e `Patient` autenticados?
- As métricas obrigatórias de cada relatório estão presentes e matematicamente corretas — os totais batem, as categorias estão completas?
- O filtro por intervalo de datas funciona corretamente — `startDate` posterior a `endDate` retorna `400`?
- A fórmula de taxa de ocupação está claramente definida no relatório e implementada de forma consistente — o número retornado corresponde à fórmula documentada?
- A estrutura de resposta dos relatórios é clara e consistente entre os quatro *endpoints* — sem formatos diferentes para cada relatório?
- Os relatórios não suportam parâmetros de paginação — o sistema ignora ou rejeita `page` e `limit` de forma adequada?
- Os dados são consultados em tempo real — inserir novos registros no banco e chamar o relatório novamente reflete as mudanças?

Penalizações proporcionais: relatórios ausentes, métricas incorretas ou incompletas, fórmula de taxa de ocupação não documentada ou inconsistente com a implementação, controle de acesso ausente, estrutura de resposta inconsistente entre relatórios e filtros de data não funcionando resultam em desconto proporcional dentro deste critério.

4. DOCUMENTAÇÃO FINAL DA API — 2,0 PONTOS

Este critério tem peso maior nesta etapa do que nas anteriores porque a documentação final representa o produto completo entregue ao longo do semestre. Uma documentação incompleta ou desatualizada na entrega final compromete a avaliação do trabalho como um todo — não apenas desta etapa.

O avaliador verificará:

Cobertura:

- Todos os *endpoints* das três etapas estão documentados — sem omissões?
- *Endpoints* adicionais implementados pelo grupo também estão documentados?
- A versão do Swagger foi atualizada para refletir a entrega final?

Autenticação:

- O botão "Authorize" está funcional — ao inserir o *token*, todos os *endpoints* protegidos o enviam automaticamente?
- Todos os *endpoints* protegidos têm `@ApiBearerAuth('access-token')`?
- O *endpoint* público `GET /reports/validate/{code}` não tem `@ApiBearerAuth()`?
- A descrição geral do Swagger explica o fluxo de autenticação de forma clara?

Hierarquias:

- Os *endpoints* com hierarquias — `POST /appointments`, `POST /appointments/{id}/procedures` — documentam os diferentes schemas com `oneOf`?
- Cada subtipo tem um exemplo distinto que ilustra seus atributos específicos — não um único exemplo genérico para todos?

Casos especiais:

- O *endpoint* de PDF documenta `Content-Type: application/pdf` e o cabeçalho `Content-Disposition`?
- O *endpoint* público de validação documenta que não requer autenticação e explica seu propósito clínico?
- Os relatórios administrativos documentam que não suportam paginação e apresentam exemplos de dados agregados realistas?

Qualidade dos exemplos:

- Nenhum exemplo usa valores genéricos — `"string"`, `0`, `true` sem contexto, tokens JWT fictícios no formato incorreto?
- Os exemplos de resposta de sucesso incluem o envelope `{ data, meta }` com `timestamp` e `path`?
- Os exemplos de resposta de erro seguem o formato RFC 7807 com `type`, `title`, `status`, `detail` e `instance`?
- Os exemplos de dados nos relatórios são internamente consistentes — os totais batem com as categorias listadas?

Consistência:

- A documentação das Etapas 1 e 2 foi revisada e está atualizada — nenhum *endpoint* tem descrição ou exemplos desatualizados em relação ao comportamento atual?
- As *tags* agrupam os *endpoints* de forma intuitiva e consistente?
- Campos sensíveis não aparecem em nenhum exemplo de resposta?

Penalizações proporcionais: *endpoints* não documentados — especialmente os das Etapas 1 e 2 —, exemplos genéricos ou inconsistentes, *endpoint* público com `@ApiBearerAuth()`, PDF sem `Content-Type` correto, hierarquias sem `oneOf`, documentação desatualizada em relação ao comportamento real e Swagger inacessível resultam em desconto proporcional dentro deste critério.

5. RELATÓRIO E DECISÕES TÉCNICAS — 1,5 PONTOS

Este critério avalia a qualidade do processo de raciocínio do grupo ao longo de toda a etapa final — e, por ser a entrega final, também a coerência do relatório como documento que acompanhou o projeto do início ao fim.

O avaliador verificará:

Completo desta etapa:

- Todas as reflexões marcadas com  ao longo deste documento foram abordadas no relatório?
- As estratégias de herança para `Appointment` e `Procedure` estão justificadas com base no perfil de consultas do domínio?
- A fórmula de taxa de ocupação está definida, justificada e consistente com a implementação?
- As operações atômicas identificadas estão descritas com a estratégia de transação adotada?
- A política de acesso a prontuários e laudos — quem pode ver o quê — está documentada e consistente

com o código?

Coerência ao longo do projeto:

- O relatório conta uma história coerente do projeto — as decisões das etapas anteriores são referenciadas quando influenciam decisões desta etapa?
- A reflexão sobre herança no banco de dados sintetiza as quatro decisões tomadas ao longo do projeto com aprendizado real — não apenas lista as estratégias escolhidas?
- A tabela final de controle de acesso cobre todos os *endpoints* das três etapas de forma consistente com o código?
- O mapa de dependências entre módulos está presente e reflete a estrutura real do código?

Limitações e reflexão:

- As limitações conhecidas do sistema estão documentadas com honestidade — de segurança, de performance, de modelagem?
- A análise de preparação para produção demonstra compreensão do que seria necessário além do que foi implementado?
- A reflexão final demonstra que o grupo compreende o que construiu, o que aprendeu e o que faria diferente?

Penalizações proporcionais: reflexões não abordadas, estratégias de herança sem justificativa baseada no domínio, fórmula de taxa de ocupação não documentada, tabela de controle de acesso incompleta ou inconsistente, mapa de dependências ausente, limitações não documentadas, ausência de reflexão final e relatório incoerente entre as etapas resultam em desconto proporcional dentro deste critério.

CONDIÇÕES DE NOTA ZERO

As situações a seguir resultam em **nota zero** para o grupo, independentemente do restante da entrega:

- **Plágio ou cópia:** código substancialmente idêntico ao de outro grupo ou obtido de fontes externas sem adaptação e sem referência. Em caso confirmado, todos os grupos envolvidos recebem nota zero.
- **Repositório inacessível:** sem acesso concedido ao professor até a data de entrega.
- **Ausência de submissão:** não submeter o link no AVA até o prazo.
- **Projeto que não inicia:** o projeto não instala ou não inicia a partir das instruções do `README.md`, impedindo qualquer avaliação.
- **Entidades centrais desta etapa não implementadas:** ausência completa das hierarquias de `Appointment` e `Procedure` — as entidades que definem o escopo desta entrega.
- **Infraestrutura das etapas anteriores ausente:** ausência de autenticação JWT, de *guards* ou de *Exception Filter* — componentes que deveriam ter sido implementados nas etapas anteriores e que são pré-requisito para esta entrega.

NOTA INDIVIDUAL

A nota é atribuída ao grupo. No entanto, se o histórico de *commits* indicar que um ou mais membros não contribuíram de forma significativa ao longo das três etapas, a nota individual desses membros poderá ser reduzida

em relação à nota do grupo, a critério do professor.

A avaliação individual desta entrega final considera o histórico completo do projeto — não apenas os *commits* da Etapa 3. Um membro que contribuiu consistentemente nas Etapas 1 e 2 mas tem ausência significativa na Etapa 3 terá sua participação avaliada de forma proporcional. O mesmo vale para o inverso — concentrar *commits* apenas na última semana do projeto, independentemente da etapa, não caracteriza desenvolvimento colaborativo.

Contribuição significativa ao longo do projeto implica: *commits* distribuídos em todas as etapas, participação ativa em revisões de *pull requests*, presença no relatório em todas as seções de contribuições individuais e *commits* que refletem trabalho real de desenvolvimento — não apenas ajustes cosméticos ou atualizações de documentação realizadas às pressas antes da entrega.